

A Project Report

on

AI-DRIVEN DEEP DOMAIN NAVIGATOR FOR INDIA

submitted in partial fulfilment of the requirements for the award of Degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING (AI&ML)

by

V. NANDINI

(22FE1A4260)

B. VARDHAN

(22FE1A4206)

K. NAGA SAI KRISHNA

(22FE1A4223)

M. YASWANTH

(22FE1A4226)

Under the Guidance of

Dr. K. VENKATESWARA RAO, Ph. D

PROFESSOR & HoD

Department of Computer Science and Engineering



VIGNAN'S LARA
INSTITUTE OF TECHNOLOGY & SCIENCE
(AUTONOMOUS)

Approved by AICTE New Delhi & Affiliated to JNTUK Kakinada
Accredited by **NAAC 'A++'** and **NBA** (CSE, IT, ECE, EEE & ME) | **ISO 9001 : 2015**
Vadlamudi - 522 213, Guntur District

April- 2026



VIGNAN'S LARA

INSTITUTE OF TECHNOLOGY & SCIENCE

(AUTONOMOUS)

Approved by AICTE New Delhi & Affiliated to JNTUK Kakinada
Accredited by **NAAC 'A++'** and **NBA** (CSE, IT, ECE, EEE & ME) | **ISO 9001 : 2015**
Vadlamudi - 522 213, Guntur District

CERTIFICATE

This is to certify that the project report entitled “**AI-Driven Deep Domain Navigator for India**” is a bonafide work done by **V. NANDINI (22FE1A4260), B. VARDHAN (22FE1A4206), K. NAGASAI KRISHNA (22FE1A4223), M. YASWANTH (22FE1A4226)** under my guidance and submitted in fulfilment of the requirements for the award of the degree of Bachelor of Technology in **COMPUTER SCIENCE AND ENGINEERING (AI&ML)** from **JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA, KAKINADA**. The work embodied in this project report is not submitted to any University or Institution for the award of any Degree or diploma.

Project Guide

Dr. K. Venkateswara Rao, Ph. D

Professor

Head of the Department

Dr. G. Rajesh Chandra, Ph. D

Professor

External Examiner

DECLARATION

We here by declare that the project report entitled “**AI-Driven Deep Domain Navigator for India**” is a record of an original work done by us under the guidance of **Dr. K. Venkateswara Rao**, Professor & HoD of Computer Science and Engineering and this project report is submitted in the fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering in Artificial Intelligence and Machine Learning. The results embodied in this project report are not submitted to any other University or Institute for the award of any Degree or Diploma.

PROJECT MEMBERS

SIGNATURE

V. NANDINI (22FE1A4260)

B. VARDHAN (22FE1A4206)

K. NAGA SAI KRISHNA (22FE1A4223)

M. YASWANTH (22FE1A4226)

Place: Vadlamudi.

Date:

ACKNOWLEDGEMENT

The satisfaction that accompanies with the successful completion of any task would be incomplete without the mention of people whose ceaseless cooperation made it possible, whose constant guidance and encouragement crown all efforts with success.

We are grateful to **Dr. K. VENKATESWARA RAO**, Professor, HoD, Department of Computer Science and Engineering for guiding through this project and for encouraging right from the beginning of the project till successful completion of the project. Every interaction with him was an inspiration.

We thank **Dr. G. RAJESH CHANDRA**, Professor, HoD, Department of Allied Computer Science and Engineering for support and valuable suggestions.

We also express our thanks to **Dr. K. PHANEENDRA KUMAR**, Principal, Vignan's Lara Institute of Technology & Science for providing the resources to carry out the project.

We also express our sincere thanks to our beloved **Chairman Dr. LAVU RATHAIAH** for providing support and stimulating environment for developing the project.

We also place our floral gratitude to all other teaching and lab technicians for their constant support and advice throughout the project.

Project Members

V. NANDINI	(22FE1A4260)
B. VARDHAN	(22FE1A4206)
K. NAGA SAI KRISHNA	(22FE1A4223)
M. YASWANTH	(22FE1A4226)

INDEX

DESCRIPTION	PAGE NUMBERS
CONTENTS	PAGE NO.
ABSTRACT	i
LIST OF FIGURES	ii
LIST OF TABLES	iii
LIST OF ABBREVIATIONS	iv
CHAPTER 1 : INTRODUCTION	1-6
1.1 Background of Domain Knowledge Exploration	3
1.2 Motivation for AI-Driven Exploration Systems	4
1.3 Overview of the Proposed AI-Driven Deep Domain Navigator	5
1.4 Scope and Significance of the System	6
CHAPTER 2 : LITERATURE SURVEY	7-12
2.1 Journals	7-10
2.2 Existing System	11
2.3 Limitations of Existing Methods	12
CHAPTER 3 : PROPOSED METHODOLOGY	13-32
3.1 Proposed System	13-15
3.1.1 Advantages of Proposed System	16-17
3.2 Methodology	18-20
3.3 Performance Evaluation and Accuracy Analysis	20-21
3.4 Domain Knowledge Structuring	21-22
3.5 Data Collection and Dataset Preparation	22
3.6 User Authentication and Profile Management	22
3.7 Machine Learning Recommendation Model	23
3.8 User Interaction and Navigation Flow	23

3.9 Building The Model	24-29
3.10 System Requirements	29-32
CHAPTER 4 : SYSTEM DESIGN	33-39
4.1 System Architecture	33
4.2 Flow Chart	34
4.3 UML Diagrams	35
4.3.1 Use Case Diagram	36
4.3.2 Class Diagram	37
4.3.3 Sequence Diagram	38
4.3.4 Activity Diagram	39
CHAPTER 5 : SYSTEM TESTING	40-42
5.1 Testing	40
5.2 Types of Tests	40-42
CHAPTER 6 : RESULTS	43-48
6.1 Comparison	43-47
6.2 Prediction	47-48
CHAPTER 7 : DEPLOYMENT	49-53
7.1 Frontend Deployment	49-50
7.2 Backend Deployment	50-51
7.3 Machine Learning Model Deployment	51
7.4 Installing and Setting up the Server	51-53
CHAPTER 8 : CONCLUSION & FUTURE WORK	54
CHAPTER 9 : REFERENCES	55-56
APPENDIX	57-80

ABSTRACT

Traditional tourism and knowledge-exploration platforms provide generalized information that is often fragmented and not tailored to individual user interests. This creates a gap in delivering meaningful, domain-specific exploration experiences where users can navigate knowledge across multiple interconnected fields such as history, architecture, culture, and ecology. Existing systems mainly rely on static content and basic search mechanisms, lacking intelligent personalization and contextual recommendations. To address this research gap, this study proposes an AI - Driven Deep Domain Navigator that enables users to explore diverse knowledge domains through an intelligent recommendation framework. The main aim of the study is to design a system that enhances user exploration by analyzing preferences and recommending relevant domains in a structured and personalized manner. The proposed system integrates web technologies with machine learning techniques to create an interactive platform. The front-end interface is developed using HTML, CSS, and JavaScript, while the backend is implemented using Node.js with MongoDB for data management. A machine learning module based on the Naïve Bayes algorithm is used to generate personalized recommendations by analyzing user interactions and preferences. The dataset used for training the model consists of categorized domain information including fields such as agriculture, architecture, art and culture, cuisine, defence, education, forestry, handlooms, history, medicine, ports, and rivers. Experimental evaluation demonstrates that the recommendation model achieves high accuracy in predicting user domain interests and delivering relevant suggestions. The results indicate that AI-based personalization significantly improves user engagement and exploration efficiency. Overall, this study contributes to society by promoting intelligent knowledge discovery, supporting digital tourism, and encouraging deeper understanding of cultural, historical, and environmental domains through personalized AI-driven exploration.

Keywords: Artificial Intelligence, Personalized Recommendation, Exploration Guide, Machine Learning, Knowledge Domains

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
3.1	Model Building Process Diagram	24
3.2	Data Pre-processing Flow Diagram	25
3.3	Model Training Process Diagram	27
3.4	Model Testing Process Diagram	28
3.5	Recommendation Process Diagram	28
4.1	User-Centric Exploration Platform Diagram	33
4.2	Overall System Architecture Diagram	33
4.3	Flow Chart Diagram	34
4.4	Use Case Diagram	36
4.5	Class Diagram	37
4.6	Sequence Diagram	38
4.7	Activity Diagram	39
6.1	Prediction Work Flow Diagram	48
7.1	Setup Development Environment Diagram	53

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
3.1	Performance Evaluation	21
3.2	Domain Categories in the System	21
3.3	Dataset Structure	25
3.4	Feature Description	26
3.5	Example Training Data	27
3.6	Hardware Requirements	30
3.7	Software Requirements	30
3.8	Development Environment	31
3.9	ML Tools	31
3.10	Network Requirements	31
3.11	Compatibility	32
6.1	Accuracy of Existing and Proposed Techniques	46
7.1	Components and Functions	52

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
ML	Machine Learning
NB	Naïve Bayes
UI	User Interface
API	Application Programming Interface
DB	Data Base
JS	Java Script
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets
NLP	Natural Language Processing

CHAPTER 1

INTRODUCTION

In the modern digital age, the availability of vast amounts of information has transformed the way people access knowledge and explore different fields. With the rapid growth of internet-based platforms, users can obtain information related to culture, history, science, environment, and many other domains within seconds. However, most existing platforms present information in a generalized and unstructured manner, which often makes it difficult for users to identify content that matches their personal interests. Users frequently encounter large volumes of data that may not be relevant to their needs, leading to confusion and reduced engagement. This challenge highlights the need for intelligent systems that can guide users through large information spaces and provide meaningful, personalized exploration experiences.

Traditional exploration and information systems mainly rely on keyword-based search or static categorization. While these approaches allow users to retrieve information, they do not effectively support personalized knowledge discovery. Users must manually browse through multiple pages or categories to find relevant content, which can be time-consuming and inefficient. Furthermore, these systems lack the ability to understand user preferences or provide context-aware recommendations. As a result, users often miss valuable information that could enhance their learning or exploration experience. This limitation creates a research gap in developing intelligent systems that can organize knowledge into meaningful domains and recommend relevant content based on user interests.

Advancements in Artificial Intelligence (AI) and Machine Learning (ML) have opened new opportunities to improve the way information systems operate. AI-based recommendation systems are capable of analyzing user behavior, preferences, and interaction patterns to generate personalized suggestions. These technologies enable systems to learn from user activities and continuously improve the quality of recommendations. By applying AI techniques, information platforms can move beyond simple search functionality and provide dynamic exploration environments where users can easily discover relevant topics across different domains.

The AI-Driven Deep Domain Navigator for India is designed to address these challenges by integrating artificial intelligence with a structured knowledge exploration platform. The main objective of this project is to create an intelligent system that allows users to explore different domains of knowledge while receiving personalized recommendations based on their interests. The system organizes information into multiple thematic domains such as agriculture, architecture, art and culture, cuisines, defence, education, forestry, handlooms, history, medicine, ports, and rivers. This domain-based structure enables users to navigate information in a more systematic and meaningful way.

The system combines modern web technologies with machine learning techniques to deliver an interactive and efficient user experience. The front-end of the application is developed using HTML, CSS, and JavaScript, which provide a user-friendly interface for navigation and interaction. The backend is implemented using Node.js, which handles server-side operations

and manages communication between the user interface and the database. MongoDB is used as the database system to store user information, feedback data, and domain-related content. This architecture ensures efficient data storage and retrieval while supporting scalability and performance.

A key component of the proposed system is the personalization engine that uses a machine learning algorithm to analyze user preferences and generate recommendations. In this project, the Naïve Bayes algorithm is used to classify and predict user interests based on their interactions with different knowledge domains. By evaluating patterns in user behavior, the system can identify which domains are most relevant to a particular user and provide suitable recommendations. This intelligent recommendation mechanism helps users discover related topics and enhances the overall exploration process.

Another important aspect of the project is its ability to promote knowledge discovery across multiple interconnected domains. Many areas of knowledge, such as history, architecture, culture, and ecology, are closely related and often influence one another. By organizing information into structured categories and enabling intelligent navigation, the proposed system encourages users to explore connections between different domains. This approach not only improves information accessibility but also supports interdisciplinary learning and awareness.

The development of the AI - Driven Deep Domain Navigator also contributes to improving digital tourism and cultural understanding. By providing structured information about various domains, users can gain insights into regional heritage, traditions, environmental resources, and historical landmarks. Such systems can help students, researchers, and travelers gain a deeper understanding of cultural and environmental aspects while promoting knowledge sharing in a digital environment.

Furthermore, the integration of artificial intelligence in exploration platforms can significantly enhance user engagement. Personalized recommendations ensure that users receive content that aligns with their interests, reducing information overload and improving the overall user experience. As users interact with the system, the recommendation engine continues to learn from their behavior, enabling the platform to deliver increasingly accurate and relevant suggestions over time.

In summary, the AI - Driven Deep Domain Navigator aims to transform traditional information exploration systems into intelligent, user-centered platforms. By combining web technologies, machine learning algorithms, and structured domain knowledge, the system provides a personalized and efficient approach to knowledge discovery. The project demonstrates how AI-driven recommendation systems can enhance digital exploration, support educational learning, and promote cultural awareness in modern information environments. Through this approach, the proposed system contributes to the development of smarter digital platforms that guide users toward meaningful and engaging knowledge exploration.

1.1 BACKGROUND OF DOMAIN KNOWLEDGE EXPLORATION

In the modern digital era, the amount of information available across various domains has increased tremendously. Knowledge related to fields such as history, architecture, agriculture, medicine, culture, and environmental studies is widely available on the internet and in digital repositories. However, this information is often scattered across multiple platforms, making it difficult for users to explore and understand a particular domain in a structured and meaningful way.

Many users, especially students and researchers, face challenges when they attempt to navigate through large volumes of unorganized data. As a result, finding relevant and meaningful information becomes time-consuming and inefficient. This problem highlights the need for intelligent systems that can organize domain knowledge and present it in a structured and user-friendly manner. Domain knowledge exploration refers to the process of discovering and navigating information within a specific subject area.

In traditional systems, users rely on simple search engines or static websites to find information. These methods typically return large lists of unrelated results, forcing users to manually filter and analyze the information to identify what is relevant to their needs. Such approaches lack contextual understanding and do not consider the user's interests, preferences, or exploration patterns. Consequently, users may miss important connections between different topics within the same domain.

Advancements in Artificial Intelligence (AI) and Machine Learning (ML) have opened new possibilities for improving how knowledge is organized and accessed. Intelligent systems can analyze user behavior, understand relationships between different topics, and provide personalized recommendations that guide users through relevant knowledge areas. By structuring domain knowledge into interconnected categories and integrating AI-based recommendation mechanisms, exploration platforms can provide a more efficient and engaging learning experience.

Therefore, developing an AI-powered domain exploration system can significantly improve the way users discover and interact with knowledge. Such systems not only help users access relevant information quickly but also encourage deeper exploration by revealing meaningful relationships between different domains. This approach enhances knowledge discovery, supports academic learning, and promotes efficient information navigation in the digital age.

Furthermore, the rapid growth of digital information has created a strong demand for intelligent knowledge management systems that can organize and present information in a meaningful way. Many existing platforms provide access to large amounts of data, but they often lack proper structure and guidance for users who wish to explore specific topics in depth. Without a clear navigation mechanism, users may struggle to understand how different concepts within a domain are related to each other. Domain knowledge exploration systems aim to solve this problem by structuring information into well-defined categories and enabling users to move easily between related topics.

1.2 MOTIVATION FOR AI-DRIVEN EXPLORATION SYSTEMS

In recent years, the rapid expansion of digital information has created new challenges for users who wish to explore knowledge across multiple domains. With the availability of vast amounts of data on the internet, users often experience information overload, where large quantities of results are returned for a single search query. Although traditional search engines and information systems provide access to a wide range of resources, they generally lack the ability to understand user preferences or guide users toward relevant knowledge paths. As a result, users must spend significant time filtering and analyzing information to identify content that matches their interests. This situation highlights the need for intelligent systems that can assist users in discovering and navigating information more effectively.

Artificial Intelligence (AI) and Machine Learning (ML) technologies have emerged as powerful tools for addressing these challenges. AI-driven systems can analyze patterns in user behavior, interpret preferences, and generate recommendations based on past interactions. By learning from user activities, such systems can provide personalized suggestions that guide users toward relevant topics and related domains. This capability is particularly useful in knowledge exploration environments where users may not always know exactly what information they are looking for but wish to discover meaningful connections between different subjects.

Another important motivation for developing AI-driven exploration systems is to improve the efficiency of knowledge discovery. Instead of presenting users with a large list of unrelated results, an intelligent exploration platform can organize information into structured domains and provide recommendations based on contextual relevance. This approach reduces the effort required to locate useful information and allows users to focus on understanding and learning rather than searching.

Furthermore, AI-powered exploration platforms can support a wide range of users, including students, researchers, educators, and general knowledge seekers. By offering personalized recommendations and structured navigation, these systems enhance the learning experience and encourage deeper exploration of knowledge domains. The integration of AI with domain-based knowledge organization therefore represents a significant step toward creating more intelligent, adaptive, and user-friendly information systems.

In addition, the motivation for developing AI-driven exploration systems also arises from the increasing need for personalized learning and intelligent guidance in digital environments. Modern users expect systems that can adapt to their interests and provide relevant suggestions without requiring extensive manual searching. Traditional information platforms treat all users in the same way, without considering individual preferences or exploration patterns. AI-driven systems, on the other hand, can analyze user interactions and continuously learn from them to improve recommendation quality. This capability allows the system to guide users toward topics that match their interests while also introducing related domains that they may not have previously considered.

1.3 OVERVIEW OF THE PROPOSED AI-DRIVEN DEEP DOMAIN NAVIGATOR

The AI-Driven Deep Domain Navigator is designed to provide an intelligent platform that enables users to explore knowledge across multiple domains in a structured and interactive manner. The system aims to address the limitations of traditional information platforms by organizing knowledge into clearly defined categories and guiding users through relevant content using artificial intelligence techniques. By integrating machine learning with web technologies, the platform provides users with a personalized exploration experience that adapts to their interests and navigation patterns.

The proposed system is developed as a web-based application that allows users to access the platform through a browser without requiring complex software installation. The system architecture consists of three main components: the frontend interface, the backend server, and the machine learning recommendation module. The frontend provides an intuitive user interface where users can register, log in, and navigate through different knowledge domains such as history, architecture, agriculture, medicine, culture, and environmental studies. Each domain contains organized information that allows users to understand and explore the subject in a systematic way.

The backend component is responsible for processing user requests, managing system operations, and communicating with the database. It is implemented using Node.js, which handles tasks such as user authentication, data processing, and communication between different modules. A MongoDB database is used to store user profiles, feedback data, and domain-related information. This ensures efficient data management and allows the system to track user interactions for further analysis.

A key feature of the proposed system is the integration of a machine learning-based recommendation engine. The system uses a Naïve Bayes algorithm to analyze user preferences and interaction patterns in order to generate personalized recommendations. When users explore different categories, the system learns their interests and suggests relevant domains that match their preferences. This intelligent recommendation mechanism helps users discover new topics and encourages deeper knowledge exploration.

Overall, the AI-Driven Deep Domain Navigator provides a modern approach to knowledge discovery by combining structured domain organization with intelligent recommendation capabilities. The system enhances the user experience by simplifying information navigation, improving accessibility to domain knowledge, and promoting efficient learning and exploration.

1.4 SCOPE AND SIGNIFICANCE OF THE SYSTEM

The proposed AI-Driven Deep Domain Navigator is designed to provide an intelligent platform that helps users explore domain-specific knowledge in an organized and interactive way. The scope of the system includes multiple knowledge domains such as history, architecture, culture, medicine, agriculture, and other related fields. Instead of presenting static information like traditional websites, the system allows users to dynamically navigate through different categories and discover related information using artificial intelligence techniques.

By structuring knowledge into organized domains and subdomains, the system enables users to access relevant information quickly and efficiently. This makes the exploration process more meaningful and user-friendly. Another important scope of the system is the integration of machine learning and intelligent recommendation mechanisms. The system analyzes user behavior, preferences, and interaction patterns to recommend relevant topics or domains.

This personalization improves the user experience and ensures that the information provided is more aligned with the user's interests. The system also supports interactive features such as user authentication, feedback collection, category-based navigation, and chatbot-based assistance. These features make the platform more engaging and allow users to interact with the system rather than just reading information passively.

The significance of the system lies in its ability to bridge the gap between large volumes of information and the user's need for structured knowledge discovery. In today's digital era, users often struggle to find accurate and relevant information from scattered sources. The proposed system organizes knowledge into clearly defined domains and provides intelligent recommendations, making the exploration process simpler and more effective. By combining artificial intelligence with a structured knowledge framework, the system promotes deeper understanding and encourages users to explore different domains of knowledge.

Furthermore, the system can serve as an educational and research support tool. Students, researchers, and knowledge seekers can use the platform to explore interdisciplinary information across multiple domains. The AI-based recommendation engine enhances learning by suggesting related topics and guiding users through meaningful exploration paths. As technology continues to advance, such intelligent exploration systems will play a significant role in digital knowledge platforms, enabling smarter access to information and improving the overall learning experience.

CHAPTER 2

LITERATURE SURVEY

2.1 JOURNALS

1. Title: Trip Craft: A Customized Journey Recommendation Bot

Authors: H. S. Murugan, K. B. Nimitha, S. S. Nimitha, and M. Rash

Abstract:

The study by H. S. Murugan, K. B. Nimitha, S. S. Nimitha, and M. Rash titled “Trip Craft: A Customized Journey Recommendation Bot” presents an intelligent travel recommendation system designed to assist users in planning personalized journeys. The proposed system utilizes artificial intelligence techniques to analyze user preferences, travel interests, and previous interactions to generate customized travel recommendations. The bot aims to simplify the travel planning process by providing suggestions related to destinations, routes, and travel activities based on user inputs. By integrating recommendation algorithms and chatbot-based interaction, the system enables users to easily communicate their travel requirements and receive relevant suggestions in a convenient and interactive manner. The research emphasizes the importance of intelligent recommendation systems in improving user experience within tourism and travel planning applications.

2. Title: HistoMind: An AI-Based Guide for Historical Sites

Authors: C. N. Ranasinghe, K. U. Ranasinghe, M. Niketh, P. Manul, H. Buddika, and R. Samantha

Abstract:

The study by C. N. Ranasinghe, K. U. Ranasinghe, M. Niketh, P. Manul, H. Buddika, and R. Samantha titled “HistoMind: An AI-Based Guide for Historical Sites” proposes an intelligent system designed to assist users in exploring historical sites using artificial intelligence techniques. The system aims to provide informative guidance about historical locations by integrating AI-based technologies that can analyze user queries and deliver relevant historical information. The platform helps tourists and researchers gain deeper insights into cultural heritage sites by presenting structured and context-aware knowledge. By utilizing AI methods, the system improves the accessibility of historical information and enhances the overall learning and exploration experience for users visiting or studying historical landmarks.

3. Title: AI-Based Intelligent Trip Planning and Recommendation System

Authors: V. Bankhele, A. Gite, A. Jadhav, O. Kholam, N. Rokde, and S. K. Said

Abstract:

The study by V. Bankhele, A. Gite, A. Jadhav, O. Kholam, N. Rokde, and S. K. Said titled “AI-Based Intelligent Trip Planning and Recommendation System” presents a smart travel planning system that utilizes artificial intelligence to assist users in organizing their trips efficiently. The proposed system analyzes user preferences such as travel interests, budget, and destination choices to generate personalized travel recommendations. By integrating machine learning techniques and data analysis, the system helps users identify suitable travel locations, plan itineraries, and optimize their travel schedules. The research highlights how AI-based recommendation systems can simplify travel planning by providing intelligent suggestions and improving the overall decision-making process for travellers. The system focuses on improving the efficiency of travel planning by automatically suggesting suitable destinations and activities based on user interests. It also helps users optimize their travel routes and schedules, making the overall planning process easier and more organized. The use of artificial intelligence allows the system to continuously learn from user interactions and improve its recommendation accuracy. This approach demonstrates how intelligent systems can enhance decision-making and provide more convenient travel planning solutions.

4. Title: Semantic Chatbots for Educational Tourism

Authors: N. Al-Sadek and P. Joshi

Abstract:

The study by N. Al-Sadek and P. Joshi titled “Semantic Chatbots for Educational Tourism” introduces a chatbot-based system designed to enhance the learning experience for tourists interested in educational and cultural destinations. The proposed system uses semantic technologies and natural language processing to understand user queries and provide meaningful information related to educational tourism sites. By integrating semantic web concepts, the chatbot is able to retrieve relevant knowledge and deliver informative responses about historical locations, museums, and cultural landmarks. The research demonstrates how conversational AI systems can support interactive learning and improve user engagement by allowing tourists to explore educational information through a simple and user-friendly communication interface. The system aims to make tourism more interactive by allowing users to ask questions and receive instant responses related to historical and educational places. The semantic structure of the chatbot helps it understand the context of user queries more accurately than traditional search systems. This improves the quality of information delivered to users and enhances their overall learning experience. The study highlights the potential of AI-powered chatbots in supporting smart tourism and educational exploration.

5. Title: Online Tourist Portals for Destination Review Aggregation

Authors: Xinxin Guo, Juho Pesonen, and Raija Komppula

Abstract:

The study by Xinxin Guo, Juho Pesonen, and Raija Komppula titled “Online Tourist Portals for Destination Review Aggregation” focuses on the role of online tourism platforms in collecting and organizing user-generated reviews about travel destinations. The research highlights how tourist portals aggregate reviews, ratings, and feedback from travelers to provide valuable insights about destinations, attractions, and travel experiences. These platforms help potential travelers make informed decisions by presenting summarized opinions and evaluations from previous visitors. The study emphasizes that review aggregation systems improve the accessibility of travel information and support tourists in comparing destinations based on real user experiences. Overall, the research demonstrates the importance of online tourist portals in enhancing destination visibility, supporting travel planning, and influencing tourist decision-making through aggregated review data. The study highlights how aggregated reviews help travelers evaluate the quality of destinations before planning their trips. Online tourist portals also allow users to compare multiple destinations based on ratings and feedback. This improves transparency in tourism services and supports better travel decision-making. The research shows that user-generated content plays an important role in shaping destination popularity and tourist preferences.

6. Title: CultureNav: AI-Based Cultural Heritage Navigation Using Semantic Knowledge Graphs

Authors: W. Li, H. Zhao, and J. Park

Abstract:

The study by W. Li, H. Zhao, and J. Park titled “CultureNav: AI-Based Cultural Heritage Navigation Using Semantic Knowledge Graphs” presents an intelligent navigation system designed to help users explore cultural heritage sites using artificial intelligence and semantic knowledge graphs. The proposed system organizes cultural and historical information into a structured knowledge graph, allowing users to discover relationships between different heritage locations, artifacts, and historical events. By using semantic technologies and AI-based data analysis, the system provides meaningful recommendations and navigation assistance to tourists and researchers. This approach improves the accessibility and understanding of cultural heritage information by presenting interconnected knowledge rather than isolated facts. The research demonstrates how AI and semantic knowledge graphs can enhance cultural tourism and support deeper exploration of historical and cultural domains. The CultureNav system enhances the tourism experience by providing context-aware information about cultural heritage sites. The knowledge graph structure allows the system to connect related historical facts, places, and cultural elements.

7. Title: Personalized Context-Aware Recommender System for Travelers

Authors: A. J. Sabet, M. Rossi, F. A. Schreiber, and L. Tanca

Abstract:

The study by A. J. Sabet, M. Rossi, F. A. Schreiber, and L. Tanca titled “Personalized Context-Aware Recommender System for Travelers” proposes a recommendation system designed to provide personalized travel suggestions based on the context and preferences of individual users. The system analyzes various contextual factors such as user location, travel history, interests, and environmental conditions to generate relevant recommendations for destinations and activities. By incorporating context-aware computing techniques, the system adapts its recommendations dynamically according to the user's situation and travel requirements. The research highlights how personalized recommendation systems can enhance the travel planning experience by providing more accurate and relevant suggestions, thereby helping travelers make better decisions and discover suitable destinations efficiently. The system continuously updates recommendations as user context or preferences change. This dynamic adaptation improves the relevance and usefulness of suggested travel options. The study demonstrates that context-aware recommendation models can significantly enhance user satisfaction and support smarter tourism planning.

8. Title: Personalized Tourism Recommendation Using Neural Collaborative Filtering

Authors: L. Yang and T. Miyazaki

Abstract:

The study by L. Yang and T. Miyazaki titled “Personalized Tourism Recommendation Using Neural Collaborative Filtering” proposes a tourism recommendation system that utilizes neural collaborative filtering techniques to provide personalized travel suggestions. The system analyzes user preferences, travel history, and interaction patterns to identify similarities between users and recommend suitable tourist destinations. By applying deep learning methods, the model is able to capture complex relationships between users and travel items, leading to more accurate and personalized recommendations. The research highlights the effectiveness of neural collaborative filtering in improving the quality of recommendation systems and enhancing the overall travel planning experience for users. The system improves recommendation accuracy by learning hidden patterns in user behavior and travel preferences. This allows the model to suggest destinations that closely match the interests of individual travelers. The study demonstrates that deep learning techniques can significantly enhance personalized tourism recommendation systems and improve user satisfaction.

2.2 EXISTING SYSTEM

In the present digital environment, users rely on various online platforms such as websites, search engines, and information portals to explore knowledge related to different domains like history, culture, tourism, education, and environment. These systems provide large collections of information that users can access through keyword-based search or category-based navigation. Although these platforms make information easily accessible, they often present content in a static and generalized manner without considering the individual preferences or interests of users.

Most existing exploration systems depend primarily on traditional search mechanisms. Users must manually enter keywords and browse through multiple search results to find relevant information. This process can be time-consuming and inefficient, especially when the available information is vast and scattered across different sources. Additionally, such systems do not provide personalized guidance, which means users often receive the same results regardless of their interests or previous interactions. As a result, users may experience information overload and difficulty in identifying meaningful or relevant content.

Another limitation of existing systems is the lack of intelligent recommendation capabilities. Many platforms display content based only on predefined categories or simple filtering methods rather than analyzing user behavior or preferences. Without advanced analytics or machine learning techniques, these systems cannot understand the user's exploration patterns or suggest related domains that might enhance their learning experience. Consequently, users miss opportunities to discover interconnected knowledge across multiple domains.

Furthermore, existing information platforms generally treat knowledge domains independently, without highlighting relationships between them. For example, topics related to history, architecture, culture, and environment are often closely connected, but traditional systems do not provide mechanisms to explore these relationships effectively. This limits the user's ability to gain a comprehensive understanding of a subject.

In addition, many current platforms lack interactive and intelligent features that improve user engagement. The absence of personalization and adaptive recommendations results in a less engaging exploration experience. Users may need to navigate through multiple pages or platforms to gather the information they require, which reduces efficiency and convenience.

Therefore, the limitations of existing systems highlight the need for an intelligent exploration platform that can organize knowledge into meaningful domains and provide personalized recommendations based on user interests. Addressing these challenges can significantly improve the efficiency of information discovery and enhance the overall exploration experience for users.

2.3 LIMITATIONS OF EXISTING METHODS

Although current digital information systems provide access to a large amount of data, they still face several limitations when it comes to intelligent exploration and personalized user experience. Most existing platforms are designed primarily to store and display information rather than guide users in discovering relevant knowledge. These systems often rely on static content structures and basic search mechanisms, which do not consider the individual interests, behaviour, or preferences of users. As a result, users must spend significant time browsing through multiple sources to locate useful information.

Another major limitation of existing methods is the lack of personalization. Many platforms provide the same set of information to every user regardless of their background, interests, or previous interactions. This approach fails to create a meaningful exploration experience because users are not guided toward content that matches their preferences. In addition, traditional systems do not effectively analyze user interaction patterns or learning behavior, which prevents them from offering intelligent recommendations or domain-based suggestions.

Existing exploration platforms also struggle to handle relationships between different knowledge domains. Information related to fields such as culture, history, architecture, and environment is often interconnected, but traditional systems treat them as separate entities. Without an intelligent mechanism to link related domains, users may miss valuable insights that could enhance their understanding. This limitation reduces the overall effectiveness of digital knowledge exploration systems.

Furthermore, many current platforms lack adaptive learning capabilities. They do not update or improve recommendations based on user feedback or behavior over time. Because of this, the systems remain static and cannot evolve to provide better results or suggestions for users. These limitations highlight the need for an advanced system that integrates artificial intelligence and machine learning techniques to provide personalized and dynamic exploration experiences.

Major limitations of existing methods include:

1. Lack of personalized recommendations based on user interests and preferences.
2. Heavy dependence on keyword-based search rather than intelligent exploration.
3. Inability to analyze user behavior and interaction patterns effectively.
4. Poor integration of related knowledge domains.
5. Limited user engagement due to static and non-adaptive content.
6. Difficulty in discovering relevant information from large data sources.
7. Absence of machine learning techniques for improving recommendation accuracy.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 PROPOSED SYSTEM

To overcome the limitations of traditional exploration platforms, the proposed AI - Driven Deep Domain Navigator introduces an intelligent system that integrates artificial intelligence, machine learning, and modern web technologies to provide a personalized knowledge exploration experience. Unlike existing systems that rely only on static content and keyword-based searches, the proposed system analyzes user preferences and interactions to deliver relevant and meaningful recommendations. The platform organizes knowledge into structured domains and allows users to explore interconnected topics efficiently. By incorporating a machine learning-based recommendation engine, the system can predict user interests and suggest related domains, improving the overall exploration process. The proposed system also provides an interactive user interface and efficient data management to ensure smooth navigation and effective knowledge discovery. Through this approach, the system aims to enhance user engagement, reduce information overload, and support intelligent exploration across multiple knowledge domains.

i. Domain-Based Knowledge Organization

One of the main components of the proposed system is the organization of information into structured knowledge domains. Instead of presenting scattered information, the system categorizes content into clearly defined domains such as agriculture, architecture, art and culture, cuisines, defence, education, forestry, handlooms, history, medicine, ports, and rivers. This structured organization allows users to explore information in a systematic and meaningful way.

By grouping related topics under specific domains, the system helps users easily navigate through large amounts of information. It also highlights the relationships between different knowledge areas, encouraging interdisciplinary learning and exploration. For example, topics related to culture may also connect with history, architecture, and traditional crafts, enabling users to gain a broader understanding of the subject.

The domain-based structure improves accessibility and simplifies the exploration process for users with different interests. Instead of searching randomly, users can directly access the domain that matches their curiosity and explore relevant content within that category.

Key features of domain-based organization include:

- **Structured categorization of information** - The system organizes knowledge into well-defined domains, making it easier for users to locate and explore relevant topics.
- **Improved navigation** - Users can quickly access specific domains and explore related subtopics without browsing through unrelated content.

- **Interconnected knowledge discovery** - By linking related domains, the system allows users to explore relationships between different fields of knowledge.

ii. AI-Based Personalization Engine

The core innovation of the proposed system is the integration of an artificial intelligence–based personalization engine. This component analyzes user behavior, preferences, and navigation patterns to generate personalized recommendations. By applying machine learning techniques, the system learns from user interactions and continuously improves the accuracy of its suggestions.

The personalization engine evaluates how users interact with different domains and identifies patterns that indicate their interests. Based on this analysis, the system recommends related domains or topics that the user may find useful or interesting. This intelligent recommendation process helps users discover new areas of knowledge while maintaining relevance to their preferences.

The system uses the Naïve Bayes machine learning algorithm to classify user interests and generate predictions. This algorithm analyzes interaction data and determines the probability of a user being interested in a particular domain. The use of machine learning ensures that the recommendations become more accurate as more user data is collected.

Important aspects of the AI personalization engine include:

- **User behavior analysis** - The system tracks user interactions such as domain visits and preferences to understand their interests.
- **Machine learning–based recommendation** - The Naïve Bayes algorithm predicts relevant domains and suggests personalized exploration paths.
- **Continuous improvement** - As the system gathers more interaction data, the recommendation accuracy improves over time.

iii. Web-Based Interactive Platform

The proposed system is implemented as an interactive web platform that allows users to explore different knowledge domains easily. The platform provides a simple and user-friendly interface that enables smooth navigation between categories, personalized recommendations, and feedback features. The user interface is designed to be accessible and visually organized so that users can quickly understand the available domains and explore them effectively.

The front-end of the system is developed using HTML, CSS, and JavaScript, which provide a responsive and interactive interface for users. These technologies enable dynamic page updates, navigation menus, and structured presentation of domain information. The backend is implemented using Node.js, which manages server-side processing and handles communication between the front-end and the database.

MongoDB is used as the database system to store user information, domain content, and feedback data. This combination of technologies ensures efficient data storage, faster processing, and smooth interaction between different components of the system.

Key features of the web-based platform include:

- **User-friendly interface** - The platform provides simple navigation menus and domain categories to make exploration easy for users.
- **Efficient data management** - MongoDB stores domain information and user data, enabling quick retrieval and updates.
- **Interactive exploration environment** - Users can explore domains, receive recommendations, and interact with the system seamlessly.

iv. User Interaction and Feedback Mechanism

User interaction and feedback play an important role in improving the performance of the proposed system. The platform allows users to provide feedback about their exploration experience, which helps the system evaluate its recommendation accuracy and effectiveness. Feedback data is stored in the database and used to analyze user satisfaction and system performance.

By collecting feedback from users, the system can identify areas that require improvement and adjust recommendation strategies accordingly. This mechanism ensures that the system remains dynamic and responsive to user needs. In addition, feedback helps developers understand how users interact with the platform and which domains are most frequently explored.

The integration of user feedback also supports continuous system improvement. As more feedback and interaction data are collected, the machine learning model can be updated to produce more accurate and meaningful recommendations.

Main benefits of the feedback mechanism include:

- **Improved recommendation accuracy** - User feedback helps evaluate whether the suggested domains are relevant and useful.
- **Better understanding of user needs** - Interaction data allows the system to adapt to changing user preferences.
- **Continuous system enhancement** - Feedback-driven improvements ensure that the platform evolves over time and provides better exploration experiences.

Overall, the proposed AI - Driven Deep Domain Navigator combines domain-based knowledge organization, machine learning-driven personalization, interactive web technologies, and user feedback mechanisms to create an intelligent exploration platform. By addressing the limitations of traditional information systems, the proposed solution provides a more efficient, engaging, and personalized approach to knowledge discovery.

3.1.1 ADVANTAGES OF THE PROPOSED SYSTEM

The AI - Driven Deep Domain Navigator offers several advantages compared to traditional information exploration systems. By integrating artificial intelligence, machine learning techniques, and structured knowledge domains, the proposed system improves the way users access and explore information. The system not only organizes information effectively but also provides personalized recommendations that enhance user engagement and knowledge discovery. The following points highlight the major advantages of the proposed system.

1. **Personalized User Experience** - The proposed system provides a personalized exploration experience by analyzing user interests and interaction patterns. Instead of displaying the same information to every user, the system recommends relevant domains and topics based on individual preferences. This helps users discover information that is more meaningful and useful to them.
2. **Intelligent Recommendation System** - The integration of machine learning algorithms enables the system to generate intelligent recommendations. By using the Naïve Bayes algorithm, the system can predict the domains that a user is most likely interested in. This reduces the effort required by users to manually search for relevant information.
3. **Structured Knowledge Organization** - The system organizes information into clearly defined knowledge domains such as history, architecture, culture, and environment. This structured arrangement allows users to explore topics in a systematic manner and makes it easier to navigate through different areas of knowledge.
4. **Improved Information Discovery** - Traditional systems often require users to search through multiple sources to find useful information. The proposed system simplifies this process by providing relevant recommendations and domain-based navigation. As a result, users can quickly discover valuable information without spending excessive time searching.
5. **Enhanced User Engagement** - Personalized recommendations and an interactive interface encourage users to spend more time exploring the platform. When users receive content that matches their interests, they are more likely to continue exploring related topics, which increases engagement and satisfaction.
6. **Efficient Data Management** - The system uses a database to store user information, feedback data, and domain content efficiently. This allows quick retrieval and updating of information, ensuring smooth system performance and faster response times during user interactions.
7. **Support for Interdisciplinary Learning** - Many knowledge domains are interconnected, and understanding one domain often requires knowledge from related fields. The proposed system allows users to explore relationships between different domains, encouraging interdisciplinary learning and broader knowledge exploration.

8. Scalability and Flexibility - The system architecture is designed to support future expansion. New domains, datasets, or recommendation algorithms can be easily integrated into the system without major modifications. This flexibility ensures that the platform can grow and adapt to new requirements.

9. Continuous System Improvement - Through user interaction and feedback mechanisms, the system continuously learns and improves its recommendations. As more users interact with the platform, the machine learning model becomes more accurate in predicting user interests, leading to better exploration results.

10. Contribution to Digital Knowledge and Cultural Awareness - The system helps promote awareness of various cultural, historical, and environmental domains by presenting them in an organized and accessible way. It supports digital learning, encourages exploration of diverse knowledge areas, and contributes to the preservation and dissemination of valuable information in society.

11. Time Efficiency - The proposed system helps users save time by providing direct access to relevant information through personalized recommendations. Instead of browsing multiple websites or pages, users can quickly find topics related to their interests within the platform. This improves the efficiency of the knowledge exploration process.

12. User-Friendly Interface - The platform is designed with a simple and intuitive interface that allows users to navigate easily between different domains and features. Clear menus, structured categories, and interactive elements help users explore information without technical difficulties, making the system accessible to a wide range of users.

13. Reduced Information Overload - Modern information platforms often contain excessive amounts of data, which can overwhelm users. The proposed system filters and presents only the most relevant content based on user preferences. This reduces unnecessary information and allows users to focus on meaningful knowledge.

14. Support for Educational and Research Activities - The system can be useful for students, researchers, and educators who want to explore domain-based knowledge efficiently. By organizing information into structured categories and providing intelligent recommendations, the platform supports academic learning, research exploration, and knowledge sharing.

15. Easy Integration with Future Technologies - The proposed system is designed in a modular way, which allows easy integration with future technologies such as advanced AI models, chatbots, maps, or multimedia galleries. This ensures that the system can evolve with technological advancements and provide more advanced exploration features in the future.

3.2 METHODOLOGY

The methodology of the AI - Driven Deep Domain Navigator focuses on developing an intelligent system that allows users to explore knowledge domains in a personalized and structured manner. The system integrates web technologies, database management, and machine learning techniques to provide efficient knowledge discovery. The methodology includes several stages such as system design, data preparation, machine learning implementation, user interaction analysis, and performance evaluation. Each stage contributes to building a platform that can recommend relevant domains based on user interests and improve exploration efficiency.

The first step in the methodology is the design of the overall system architecture. The architecture defines how different components of the system interact with each other. The system consists of a front-end interface, a backend server, a database, and a machine learning module. The front-end interface is responsible for providing an interactive environment where users can browse different domains and view recommendations. The backend server handles data processing, communication between components, and system operations. The database stores user information, domain content, and feedback data. The machine learning module analyses user interactions and generates personalized recommendations.

After designing the system architecture, the next stage involves organizing knowledge into structured domains. Instead of storing information randomly, the system categorizes data into multiple domains such as agriculture, architecture, art and culture, cuisines, defence, education, forestry, handlooms, history, medicine, ports, and rivers. This structured organization helps users explore topics in a systematic manner and makes it easier for the machine learning model to analyse relationships between domains. Domain-based organization also supports efficient data management and improves the overall navigation experience.

Another important stage of the methodology is data preparation. The dataset used in the system contains information related to various domains along with domain categories and associated attributes. This dataset acts as the training data for the machine learning model. The data is pre-processed to ensure consistency and accuracy before being used for model training. Data pre-processing may include cleaning the data, organizing domain labels, and converting information into a format suitable for machine learning analysis.

The machine learning component is implemented using the Naïve Bayes algorithm, which is widely used for classification and recommendation tasks. The algorithm analyses user interaction patterns and predicts which domain a user is most likely interested in. When users explore different categories or interact with domain pages, the system records these interactions and uses them as input for the machine learning model. Based on this information, the algorithm calculates probabilities and generates recommendations that match the user's preferences. The use of machine learning enables the system to continuously improve its recommendation accuracy as more user interaction data becomes available.

User interaction plays a crucial role in the methodology of the proposed system. The platform allows users to register, log in, explore domain categories, and provide feedback. As users interact with the system, their behaviour is recorded and analysed to identify patterns in domain exploration. This information helps the system understand user interests and recommend related domains. The interactive features also improve user engagement and make the exploration process more effective.

The feedback mechanism is another important part of the methodology. Users are provided with the option to submit feedback about the platform and their exploration experience. Feedback data is stored in the database and analysed to evaluate the effectiveness of the recommendation system. This mechanism helps developers identify potential improvements and refine the system over time.

Finally, the performance of the machine learning model is evaluated to measure its effectiveness. The evaluation process analyses the accuracy of domain predictions and recommendation results. Accuracy metrics are calculated by comparing predicted domains with actual user interests based on interaction data. The results demonstrate that the recommendation model provides reliable and relevant suggestions, improving the overall exploration experience for users.

Overall, the proposed methodology integrates structured knowledge organization, machine learning-based recommendations, and user interaction analysis to create an intelligent exploration platform. By combining these components, the system provides a more efficient and personalized approach to knowledge discovery compared to traditional exploration platforms.

The proposed methodology also focuses on creating a scalable and flexible exploration platform that can adapt to the changing needs of users. The system is designed in a modular manner so that each component can function independently while still interacting efficiently with other modules. This modular architecture allows developers to improve or upgrade specific components such as the recommendation engine, database system, or user interface without affecting the overall functionality of the platform. Such flexibility is important for modern digital systems where continuous improvements and updates are required.

Another key aspect of the methodology is the integration of both user-centred design and intelligent automation. The system is built with the objective of making knowledge exploration simple and engaging for users from different backgrounds. The interface allows users to easily browse domains, view recommended topics, and interact with different sections of the platform. At the same time, the machine learning component works in the background to analyse user behaviour and generate meaningful recommendations. This combination of human-centred design and automated intelligence improves both usability and efficiency.

The system also emphasizes effective data handling and management. All domain information, user data, and feedback records are stored in a centralized database to ensure consistency and

easy retrieval. Proper data organization helps the system process large amounts of information without affecting performance. It also enables the machine learning module to access relevant data for training and prediction tasks. By maintaining structured datasets, the system ensures that the recommendation model produces accurate and reliable results.

3.3 Performance Evaluation and Accuracy Analysis

The performance of the AI - Driven Deep Domain Navigator is evaluated to measure how effectively the system recommends relevant knowledge domains to users. The evaluation mainly focuses on the accuracy of the machine learning model used in the recommendation engine. In this system, the Naïve Bayes classification algorithm is used to analyse user interaction data and predict the most relevant domains based on user preferences. Performance evaluation helps determine whether the system is capable of generating reliable and meaningful recommendations for users.

The evaluation process begins by collecting interaction data from users when they explore different domains within the platform. This interaction data includes information such as selected categories, navigation behaviour, and feedback provided by users. The machine learning model uses this information as input to predict the domain that best matches the user's interests. The predicted results are then compared with the actual user preferences to measure the performance of the system.

To evaluate the effectiveness of the recommendation model, standard classification metrics such as Accuracy, Precision, Recall, and F1-Score can be used. Among these metrics, accuracy is the primary metric used in this system to measure the correctness of domain predictions.

Accuracy Formula:

The accuracy of the model is calculated using the following formula:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

- **TP (True Positive)** – Correctly predicted relevant domains
- **TN (True Negative)** – Correctly predicted irrelevant domains
- **FP (False Positive)** – Incorrectly predicted relevant domains
- **FN (False Negative)** – Incorrectly predicted irrelevant domains

Accuracy represents the percentage of correct predictions made by the machine learning model compared to the total number of predictions.

Table 3.1: Performance Evaluation

Evaluation Metric	Value
Accuracy	97%
Precision	96.08%
Recall	94.56%

3.4 Domain Knowledge Structuring

Domain Knowledge Structuring is an important part of the proposed methodology in the AI-Driven Deep Domain Navigator. In this system, knowledge is organized into clearly defined domains so that users can easily explore different areas of information in a structured and meaningful manner. Instead of presenting large amounts of unorganized data, the system categorizes information into specific domains such as agriculture, architecture, art and culture, cuisines, defence, education, forestry, handlooms, history, medicine, ports, and rivers. This structured organization helps users navigate the platform efficiently and discover information that is relevant to their interests.

The domain knowledge structuring process begins by identifying important thematic areas that represent different aspects of knowledge. Each domain contains related information and topics that belong to a particular category. By grouping similar information together, the system reduces complexity and improves the accessibility of knowledge.

Another advantage of domain structuring is that it supports better understanding of relationships between different knowledge areas. Many domains are interconnected; for example, history is often related to culture, architecture, and traditional crafts. By organizing information into structured domains, the system can highlight these relationships and encourage users to explore related topics.

Table 3.2: Domain Categories in the System

Domain ID	Domain Name
D1	Agriculture
D2	Architecture
D3	Art and Culture
D4	Cuisines
D5	Defence
D6	Education
D7	Forestry
D8	Handlooms
D9	History
D10	Medicine
D11	Ports
D12	Rivers

Each domain contains a collection of information related to its topic. Users can select any domain and explore the available content within that category. The system also uses this domain structure as input for the recommendation engine to generate personalized suggestions.

Overall, domain knowledge structuring plays a crucial role in improving the organization, accessibility, and exploration of information within the system. By categorizing knowledge into well-defined domains and establishing relationships between them, the proposed system creates a more efficient and user-friendly environment for digital knowledge discovery.

3.5 Data Collection and Dataset Preparation

Data collection and dataset preparation play an important role in the development of the AI - Driven Deep Domain Navigator. The system requires a well-organized dataset that contains information related to different knowledge domains so that the machine learning model can analyse and generate accurate recommendations. In this project, domain-related information is collected from reliable digital sources such as educational resources, knowledge portals, and publicly available datasets related to culture, history, environment, and other thematic areas.

After collecting the data, it is organized into structured domain categories such as agriculture, architecture, art and culture, cuisines, defence, education, forestry, handlooms, history, medicine, ports, and rivers. Each domain contains specific information that represents its category. The collected data is then cleaned and pre-processed to remove inconsistencies, duplicate entries, or incomplete information. Data pre-processing ensures that the dataset is consistent, accurate, and suitable for use in the machine learning model.

3.6 User Authentication and Profile Management

User authentication and profile management are essential components of the proposed system as they enable secure access to personalized features. The system allows users to register and create individual accounts using their personal details. During registration, user information is stored securely in the database. Once registered, users can log in to the platform using their credentials to access the system and explore different knowledge domains.

Authentication ensures that only authorized users can access the system and interact with personalized features such as recommendations and feedback submission. The login mechanism verifies the user's identity before granting access to the platform. This process helps maintain data security and prevents unauthorized access to user information.

Profile management also allows the system to maintain a record of user interactions and preferences. As users explore different domains, the system tracks their activity and stores interaction data in the database. This information is used by the machine learning recommendation engine to analyze user interests and generate personalized suggestions. By maintaining individual user profiles, the system can provide a customized exploration experience that improves user engagement and knowledge discovery.

3.7 Machine Learning Recommendation Model

The Machine Learning Recommendation Model is one of the key components of the AI Powered Personalized Exploration Guide. It is responsible for analysing user preferences and generating personalized recommendations for knowledge domains. In this system, the Naïve Bayes classification algorithm is used as the machine learning model to predict the domains that are most relevant to a user. Naïve Bayes is widely used in recommendation and classification tasks because of its simplicity, efficiency, and ability to handle large datasets.

The recommendation process begins when users interact with the system by exploring different domain categories. The system records these interactions and stores them as user behaviour data in the database. This interaction data acts as input for the machine learning model. The Naïve Bayes algorithm analyses the probability of a user being interested in a particular domain based on previous interactions and preferences. By calculating these probabilities, the model identifies the most suitable domains and recommends them to the user.

The machine learning model continuously improves its prediction capability as more interaction data becomes available. As users explore additional domains, the system gathers more information about their interests. This allows the recommendation engine to refine its predictions and provide more accurate suggestions over time. The use of a machine learning approach ensures that the system adapts to user behaviour and delivers a personalized exploration experience.

3.8 User Interaction and Navigation Flow

User interaction and navigation flow describe how users interact with the platform and explore different knowledge domains within the system. The system is designed with a simple and intuitive interface so that users can easily navigate through various sections of the platform. After logging into the system, users are presented with a homepage that displays multiple domain categories. These categories allow users to select and explore topics that match their interests.

When a user selects a domain, the system displays detailed information related to that category. Users can navigate between different domains using the navigation menu or explore recommended domains suggested by the machine learning model. The system tracks user navigation patterns, including the domains visited and the frequency of interactions. This information is used by the recommendation engine to analyse user preferences.

The platform also provides features such as feedback submission and user account management, which further enhance interaction with the system. A well-structured navigation flow ensures that users can move smoothly between different sections of the platform without confusion. By combining user-friendly navigation with intelligent recommendations, the system creates an engaging and efficient exploration environment for discovering domain-based knowledge.

3.9 Building The Model

Building the model is a critical stage in the development of the AI - Driven Deep Domain Navigator. The objective of the model is to analyze user interaction data and generate personalized recommendations for knowledge domains. The system uses a Machine Learning–based classification approach to identify user interests and suggest relevant domains.

i. Overview of the Model

The model is developed using the Naïve Bayes algorithm, which is widely used in classification problems due to its simplicity, computational efficiency, and strong performance with structured datasets. In the proposed system, the model learns patterns from user interaction data such as visited domains, preferences, and navigation behavior. Based on these patterns, the system predicts the domains that the user is most likely interested in exploring.

The model building process involves several steps including data collection, data preprocessing, feature extraction, model training, model testing, and performance evaluation. Each of these steps contributes to improving the accuracy and reliability of the recommendation system.

ii. Model Development Workflow

The overall workflow of the model development process is shown below.

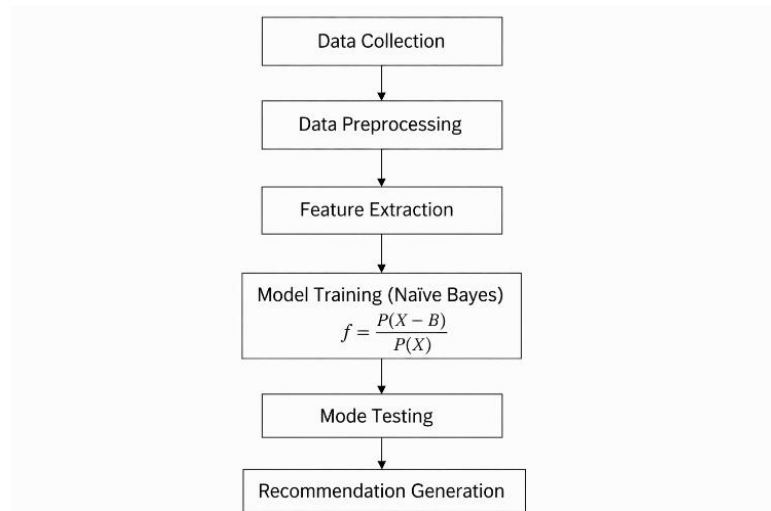


Fig 3.1 : Model Building Process Diagram

This workflow ensures that the data is properly prepared before training the machine learning model and that the model is evaluated to measure its performance.

iii. Data Collection for Model Training

The first step in building the model is collecting relevant data. The dataset used in this project contains information about different knowledge domains and user interaction patterns. The dataset helps the model understand the relationship between user behavior and domain preferences.

Table 3.3: Dataset Structure

Attribute	Description
User ID	Unique identifier for each user
Domain Selected	Domain category selected by the user
Interaction Frequency	Number of times a domain is visited
User Preference	Indicates user interest in a domain
Feedback Rating	User feedback for recommendations

The dataset is stored in the system database and used for training the machine learning model.

iv. Data Pre-processing

Raw data collected from user interactions may contain missing values, duplicate records, or inconsistent information. Therefore, data pre-processing is necessary to prepare the dataset for machine learning analysis.

Data pre-processing involves several steps:

- Removing duplicate entries
- Handling missing values
- Organizing domain labels
- Converting categorical data into numerical format
- Structuring the dataset for training

These steps ensure that the dataset is clean and suitable for training the machine learning model.

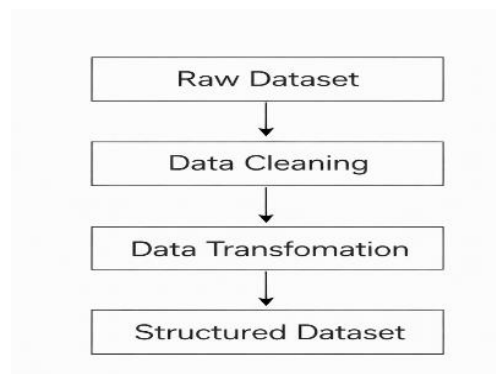


Fig 3.2: Data Pre-processing Flow Diagram

v. Feature Extraction

Feature extraction is the process of selecting important attributes from the dataset that will help the machine learning model make predictions. In the proposed system, features are derived from user interaction data and domain preferences.

Important features used in the model include:

- Domain category selected by the user
- Number of visits to a domain
- User feedback rating
- Interaction frequency
- Historical user preferences

These features help the Naïve Bayes algorithm determine the probability that a user is interested in a particular domain.

Table 3.4: Feature Description

Feature Name	Description
Domain Category	Type of Domain Explored
Visit Frequency	Number of visits to a domain
Interaction Time	Time spent exploring a domain
Feedback Score	Rating provided by user
Preference Label	Indicates user interest level

Feature extraction improves the ability of the model to learn patterns in user behavior.

vi. Naïve Bayes Algorithm

The proposed system uses the Naïve Bayes classification algorithm to predict user interests. Naïve Bayes is a probabilistic algorithm based on Bayes' theorem, which calculates the probability of an event occurring based on prior knowledge.

Bayes Theorem Formula:

$$P\left(\frac{A}{B}\right) = \frac{P\left(\frac{B}{A}\right) * P(A)}{P(B)}$$

Where:

- **P(A|B)** – Probability of event A given event B
- **P(B|A)** – Probability of event B given event A

- **P(A)** – Prior probability of event A
- **P(B)** – Probability of event B

In the context of the exploration guide:

- A represents a **user's interest in a domain**
- B represents **user interaction data**

The Naïve Bayes algorithm assumes that features are independent of each other, which simplifies the calculation and makes the model efficient for classification tasks.

vii. Model Training

Model training is the process of teaching the machine learning algorithm to recognize patterns in the dataset. During training, the Naïve Bayes algorithm analyzes the dataset and calculates probabilities for each domain based on user interactions.

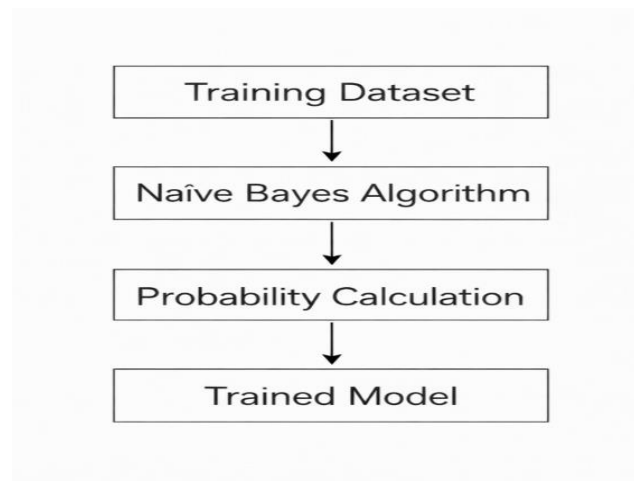


Fig 3.3: Model Training Process Diagram

The trained model can then predict the most relevant domain for a user based on their behavior.

Table 3.5: Example Training Data

User	Domain Visited	Preference
U1	History	High
U2	Architecture	Medium
U3	Culture	High
U4	Education	Low

The algorithm uses this data to learn patterns and classify user interests.

viii. Model Testing

After training the model, it is tested using a separate dataset to evaluate its performance. The testing phase ensures that the model can correctly predict user interests for new interaction data.

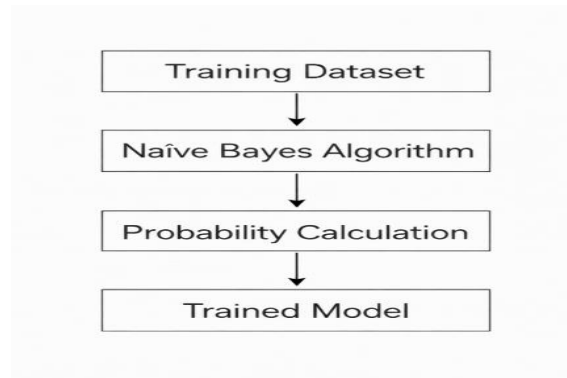


Fig 3.4: Model Testing Process Diagram

The predicted results are compared with the actual user preferences to determine the model's accuracy.

ix. Recommendation Generation

Once the model is trained and tested successfully, it is integrated into the exploration platform to generate personalized recommendations.

When a user interacts with the system:

1. The system records user interaction data
2. The trained model analyzes the data
3. The model predicts relevant domains
4. Recommended domains are displayed to the user

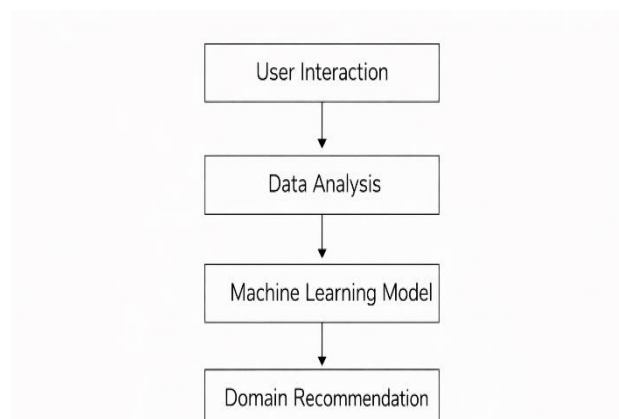


Fig 3.5: Recommendation Process Diagram

x. Advantages of the Model

The machine learning model provides several benefits:

- Provides personalized domain recommendations
- Improves user exploration experience
- Learns from user interaction data
- Adapts to changing user preferences
- Enhances accuracy of knowledge discovery

Building the machine learning model is an essential step in the development of the AI - Driven Deep Domain Navigator. By combining structured datasets, feature extraction, and the Naïve Bayes classification algorithm, the system can analyze user behavior and generate meaningful recommendations. The model training and evaluation processes ensure that the recommendations are accurate and reliable. As more interaction data is collected, the system continues to improve its prediction capability, providing users with a more personalized and efficient exploration experience.

3.10 System Requirements

i. Introduction to System Requirements

System requirements define the minimum hardware and software environment required for the successful execution of the AI-Driven Deep Domain Navigator system. These requirements ensure that the system operates efficiently, maintains stability, and provides accurate recommendations to users.

The project integrates multiple technologies including web development, database management, and machine learning components. Therefore, the system requires a combination of development tools, runtime environments, and supporting software.

The system requirements are categorized into two main groups:

- Hardware Requirements
- Software Requirements

These requirements ensure proper functioning of the application including user authentication, data storage, machine learning model processing, and recommendation generation.

ii. Hardware Requirements

Hardware requirements refer to the physical components required to develop, run, and test the application.

The system does not require high-end computing hardware because the machine learning model used is lightweight and efficient. However, a basic system configuration is required for development and execution.

Table 3.6: Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
Processor	Intel i3 / AMD Equivalent	Intel i5 or higher
RAM	4 GB	8GB
Storage	20 GB Free Space	50 GB Free Space
Internet	Broadband Connection	High-Speed Internet
Device	Laptop/Desktop	Laptop/Desktop

The hardware resources are mainly required for running the web server, executing machine learning algorithms, and storing the application data.

iii. Software Requirements

Software requirements include the programming environments, tools, frameworks, and libraries needed to develop and run the project.

The system uses a full-stack web development environment combined with machine learning tools.

Table 3.7: Software Requirements

Software	Purpose
Node.js	Backend server development
HTML, CSS, JavaScript	Frontend interface
MongoDB Atlas	Cloud database storage
Python	Machine learning model
Visual Studio Code	Code editor
GitHub	Version control and project hosting
Web Browser (Chrome/Edge)	Running and testing the application

These tools allow seamless communication between the frontend, backend server, and database components.

iv. Development Environment

The development environment includes the tools and platforms used during the implementation phase of the project.

Table 3.8: Development Environment

Tool	Description
Visual Studio Code	Used for writing and editing project code
Node Package Manager (NPM)	Used to install required backend libraries
MongoDB Atlas	Cloud database used to store user data
GitHub	Used for code management and collaboration
Live Server Extension	Used for testing frontend pages

The development environment helps developers build, test, and debug the system efficiently.

v. Machine Learning Environment

Since the project includes a recommendation system using Naïve Bayes, a machine learning environment is required.

These tools help train the recommendation model and evaluate the prediction accuracy.

Table 3.9: ML Tools

Tool	Function
Python	Programming language used for ML
Scikit-learn	Library used for implementing Naïve Bayes algorithm
Pandas	Used for dataset processing
Numpy	Used for numerical computations
Matplotlib	Used for visualization and evaluation

vi. Network Requirements

Since the application uses cloud database services and web technologies, a stable internet connection is necessary.

Table 3.10: Network Requirements

Parameter	Requirement
Internet Speed	Minimum 10 Mbps
Protocol	HTTP / HTTPS
Cloud Access	MongoDB Atlas
Browser Support	Chrome, Edge, Firefox

Reliable network connectivity ensures smooth communication between the frontend interface and backend services.

vii. Compatibility Requirements

The system is designed to work across multiple platforms and devices.

Table 3.11: Compatibility

Platform	Supported
Windows	Yes
Linux	Yes
MacOS	Yes
Android Browser	Yes
Chrome / Edge	Yes

Cross-platform compatibility ensures that users can access the system from different devices without major issues.

The system requirements play a crucial role in ensuring the successful deployment and operation of the AI-Driven Deep Domain Navigator. The combination of lightweight hardware requirements, flexible software tools, and cloud-based infrastructure makes the system scalable and easy to maintain. The integration of web technologies, database management, and machine learning models enables the system to deliver intelligent recommendations and enhance user exploration experiences. Proper system configuration ensures optimal performance, accurate recommendation generation, and smooth interaction between all components of the system.

In addition to the hardware and software configurations, the system requirements also focus on ensuring reliability, scalability, and smooth user interaction within the AI-Driven Deep Domain Navigator. The platform is designed to support multiple users simultaneously while maintaining efficient response time and accurate recommendation generation. By utilizing a cloud-based database and modular backend architecture, the system can easily scale when the number of users or stored data increases. Furthermore, the integration of the machine learning module with the web application allows the system to continuously analyze user preferences and improve recommendation quality over time. These requirements ensure that the system remains stable, flexible, and capable of delivering an intelligent exploration experience for users across different devices and environments.

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The System Architecture of the AI-Driven Deep Domain Navigator explains how different components of the system interact with each other to deliver intelligent recommendations and domain-based knowledge exploration. The architecture follows a multi-layered design consisting of the User Interface Layer, Application Layer, Machine Learning Layer, and Database Layer. Each layer performs a specific function and works together to provide a seamless and efficient user experience.

The architecture ensures that user requests move from the interface to the backend, where they are processed, analysed by the machine learning model, and then returned as personalized recommendations. This structured design improves scalability, maintainability, and performance of the system.

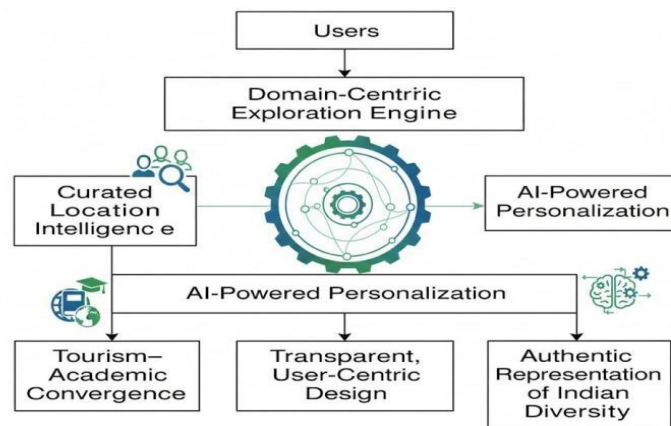


Fig 4.1: User-Centric Exploration Platform Diagram

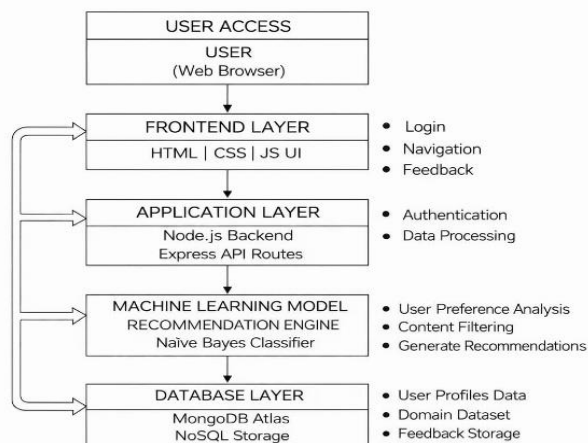


Fig 4.2: Overall System Architecture Diagram

4.2 FLOW CHART

A flow chart represents the step-by-step working process of the AI-Driven Deep Domain Navigator. It visually explains how the system processes user requests, analyzes information using machine learning, and generates personalized recommendations. Flowcharts are important in system documentation because they help readers quickly understand the sequence of operations and how different components interact within the system.

In the proposed system, the flow begins when a user accesses the platform through the web interface. The user first registers or logs into the system. After successful authentication, the user can explore different domain categories such as history, architecture, culture, or education. The system tracks user interactions and sends this data to the backend server.

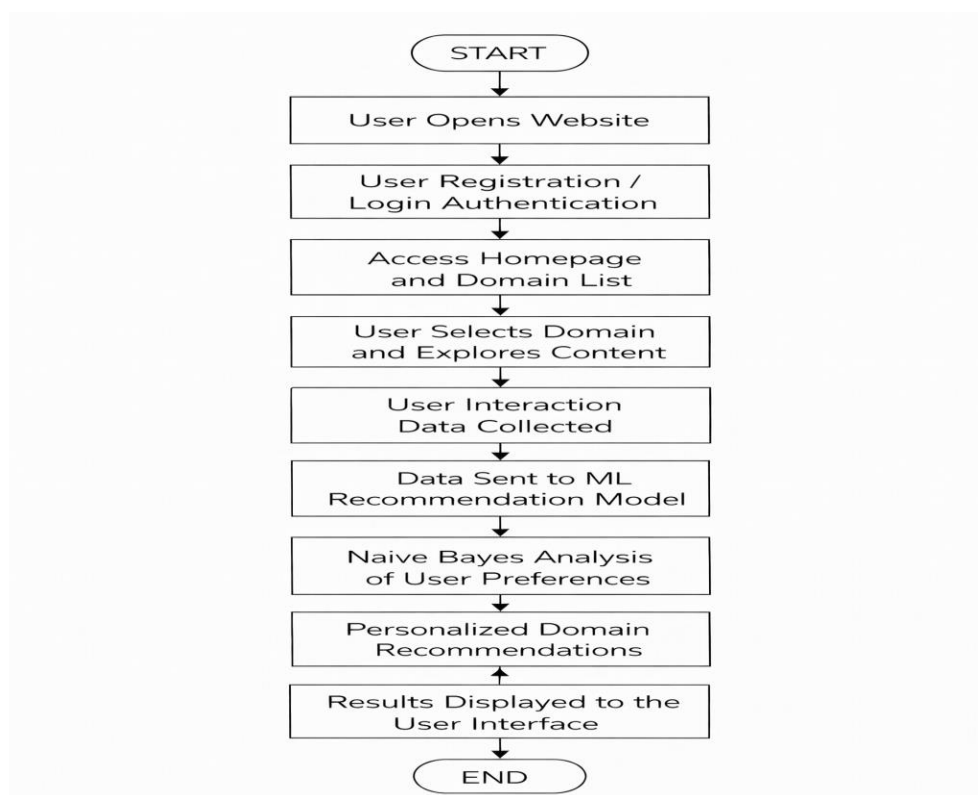


Fig 4.3: Flow Chart Diagram

The backend processes the user request and forwards the interaction data to the machine learning recommendation module. The machine learning model (Naïve Bayes) analyzes the user's navigation patterns and predicts the domains that best match the user's interests. Based on the probability results generated by the model, the system produces personalized recommendations. These recommendations are then displayed on the user interface for further exploration.

The flow chart therefore demonstrates how user input moves through different stages such as authentication, data processing, machine learning analysis, and recommendation generation before returning results to the user.

4.3 UML DIAGRAMS

Unified Modeling Language (UML) diagrams are widely used in software engineering to visually represent the structure and behavior of a system. UML provides a standardized way to model the design of software systems and helps developers understand how different components interact with each other. In the AI-Driven Deep Domain Navigator, UML diagrams are used to illustrate the relationships between users, system modules, and data processing components.

The purpose of using UML diagrams in this project is to simplify the understanding of the system architecture, user interactions, and internal processes. These diagrams help in clearly presenting the functional and structural aspects of the system to developers, researchers, and stakeholders. By representing system behavior visually, UML diagrams improve communication among team members and assist in identifying potential design improvements before implementation.

UML diagrams are generally categorized into structural diagrams and behavioral diagrams. Structural diagrams describe the static structure of the system, such as components and their relationships, while behavioral diagrams describe the dynamic interactions and workflow within the system. In this project, UML diagrams are used to represent the interaction between the user interface, backend server, machine learning model, and database

GOALS:

The main goals of including UML diagrams in this project documentation are as follows:

1. To provide a clear visual representation of the system architecture and how different components are connected.
2. To help developers, researchers, and reviewers easily understand the working of the proposed system.
3. To document the structural and behavioral design of the system in a standardized format.
4. To improve communication among project developers, instructors, and stakeholders by presenting the system workflow visually.
5. To clearly represent system requirements, user roles, and interactions within the platform.
6. To simplify complex processes such as machine learning recommendations and data flow through visual diagrams.
7. To assist developers during implementation by providing a blueprint of system functionality and structure.
8. To make it easier for future developers to understand the system design and add new features or improvements.

4.3.1 USE CASE DIAGRAM:

A Use Case Diagram is one of the most important UML diagrams used in software design to represent the interaction between users (actors) and the system. It provides a high-level view of how users interact with different features of the application. In the AI-Driven Deep Domain Navigator, the use case diagram helps illustrate how users access the system, explore domain knowledge, and receive personalized recommendations generated by the machine learning module.

The main actor in this system is the User, who interacts with the web application through a browser interface. The user performs actions such as registration, login, exploring knowledge domains, viewing recommendations, and providing feedback. Each of these interactions represents a use case, which describes a specific functionality provided by the system.

The system processes user inputs through the backend server, stores data in the database, and uses the machine learning recommendation model to analyze user preferences. Based on the interaction history, the system generates personalized recommendations and displays them on the user interface. The use case diagram visually represents these interactions and shows how different system functionalities are connected to the user

Main Actors

User

- Registers and logs into the system
- Explores different domain categories
- Receives personalized recommendations
- Provides feedback to improve the system

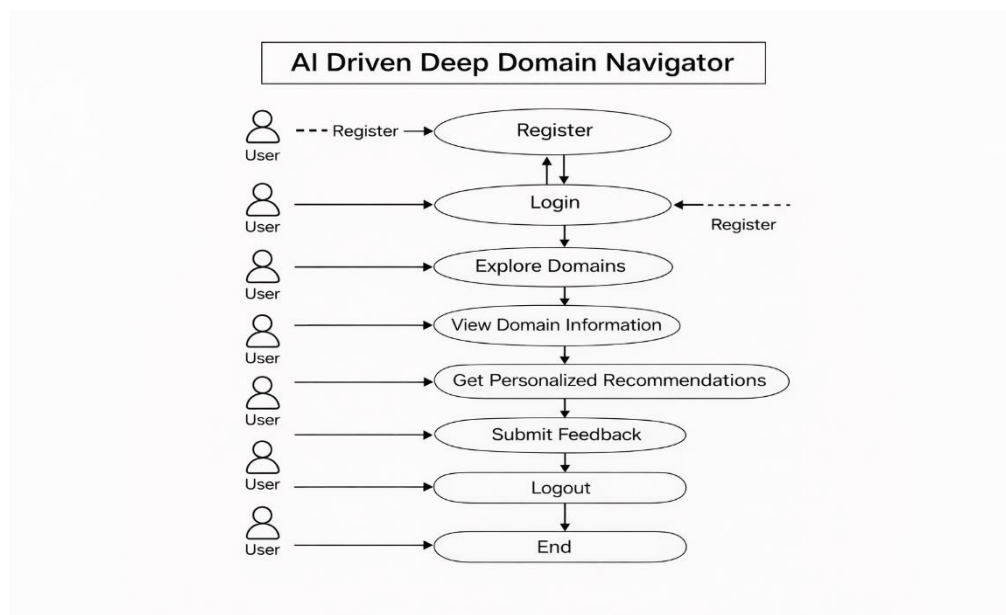


Fig 4.4: Use Case Diagram

4.3.2 CLASS DIAGRAM:

A Class Diagram is a type of UML structural diagram that represents the static structure of a system by showing its classes, attributes, methods, and the relationships between them. It helps in understanding how different components of the system are organized and how they interact with each other. In the AI-Driven Deep Domain Navigator, the class diagram illustrates the core classes involved in managing user data, domain information, machine learning recommendations, and feedback.

The system consists of several important classes such as User, Domain, Recommendation Model, Feedback, and Database. Each class contains specific attributes and functions that define its role in the system. For example, the User class manages user authentication and profile details, while the Domain class stores information related to different knowledge categories. The Recommendation Model class processes user interaction data and generates personalized suggestions using the machine learning algorithm.

The relationships between these classes demonstrate how the system operates. The User interacts with the Domain class to explore information, and the Recommendation Model analyzes user activity to generate recommendations. The Feedback class collects user responses to improve the system's accuracy, while the Database class stores all the information required for system operation. The class diagram therefore provides a clear representation of the system structure and helps developers understand how different modules interact during implementation.

Main Classes in the System:

- User Class
- Domain Class
- Recommendation Class
- Feedback Class
- Data Base Class

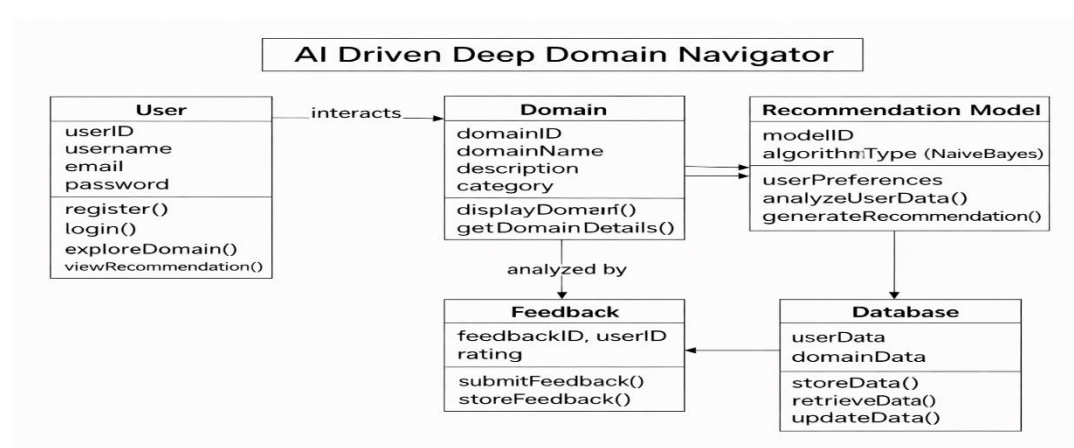


Fig 4.5: Class Diagram

4.3.3 SEQUENCE DIAGRAM:

A Sequence Diagram is a type of UML behavioral diagram that represents the interaction between different components of a system in a time-ordered sequence. It shows how objects communicate with each other and how messages are exchanged to complete a particular process. In the AI-Driven Deep Domain Navigator, the sequence diagram illustrates the flow of operations when a user interacts with the system, starting from login to receiving personalized recommendations.

The sequence diagram focuses on the dynamic behavior of the system. It describes how the User Interface, Backend Server, Database, and Machine Learning Recommendation Model interact during system execution. When a user logs into the platform, the request is sent from the user interface to the backend server. The backend verifies the credentials using the database and then grants access to the user.

After successful login, the user can explore different domain categories available in the system. Each user interaction is recorded and stored in the database. This interaction data is then processed by the machine learning recommendation model, which analyzes user preferences using the Naïve Bayes algorithm. Based on this analysis, the system generates personalized recommendations that are sent back to the backend server and displayed on the user interface.

Main Components in the Sequence Diagram:

- User
- User Interface
- Backend Server
- Database
- Recommendation Model

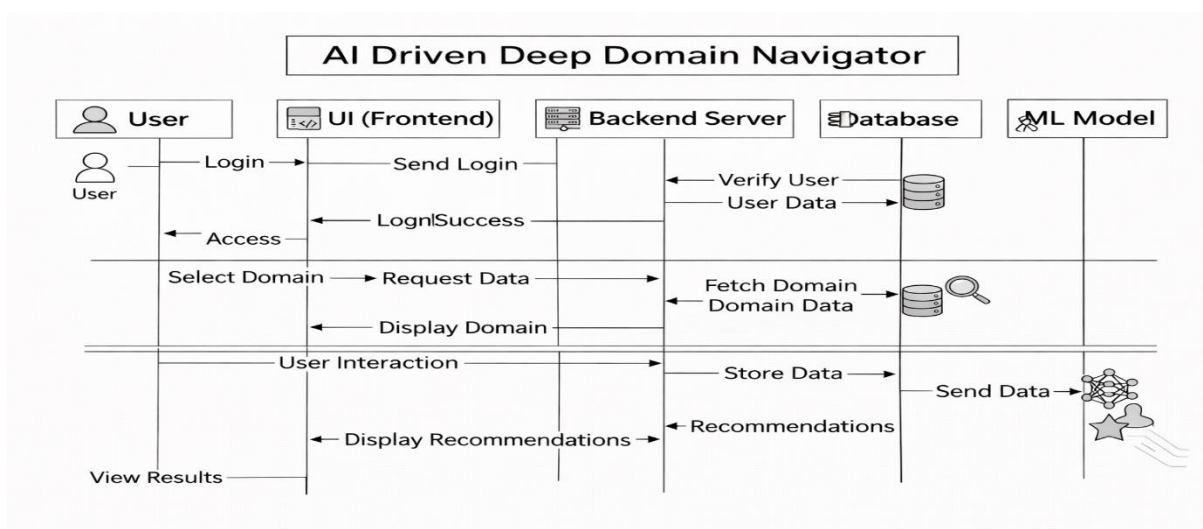


Fig 4.6: Sequence Diagram

4.3.4 ACTIVITY DIAGRAM:

An Activity Diagram is a behavioral UML diagram used to represent the workflow of activities within a system. It shows the sequence of actions, decision points, and the flow of control from the beginning to the end of a process. In the AI-Driven Deep Domain Navigator, the activity diagram illustrates the step-by-step process of how users interact with the system and how the system generates personalized recommendations based on user behavior.

The activity diagram starts when the user opens the web application and accesses the system interface. The user then proceeds to register or log in to the platform. After successful authentication, the user is directed to the homepage where different domain categories are displayed. The user can select a domain of interest and explore the information available in that category.

During the exploration process, the system continuously records the user's interactions and preferences. This interaction data is then processed by the backend server and sent to the machine learning module. The machine learning model analyzes the collected data and predicts the domains that best match the user's interests. Based on the prediction results, the system generates personalized recommendations.

Activities in the System Workflow:

- System Start
- User Authentication
- Homepage Access
- Domain Browsing
- Domain Selection

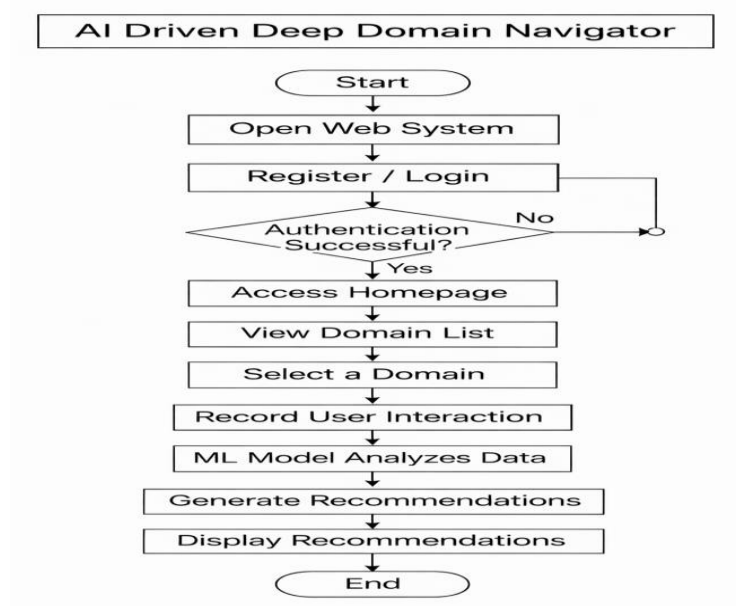


Fig 4.7: Activity Diagram

CHAPTER 5

SYSTEM TESTING

5.1 TESTING

System testing is an essential phase in software development that ensures the application works correctly and meets the expected requirements. In the AI-Driven Deep Domain Navigator, testing is performed to verify that all modules such as user authentication, domain navigation, machine learning recommendations, and database operations function properly. The purpose of testing is to detect errors, validate system performance, and ensure that the system provides accurate and reliable results to users.

Testing is carried out after the implementation of the system modules. During this phase, different components of the application are tested individually as well as collectively to ensure smooth integration. The frontend interface, backend server, database connectivity, and machine learning module are tested to confirm that they interact correctly and deliver the expected functionality. Proper testing improves the quality of the system and ensures that users can explore knowledge domains without encountering technical issues.

Another important objective of testing is to validate the recommendation model used in the system. Since the system generates personalized recommendations using the Naïve Bayes algorithm, testing is required to evaluate the accuracy of predictions. By analyzing test data and user interaction logs, the system ensures that recommendations are relevant to user interests. This helps improve user experience and enhances the reliability of the system.

5.2 TYPES OF TESTS

i. Unit Testing

Unit testing is the process of testing individual components or modules of a software application independently. Each unit is tested separately to ensure that it performs its intended function correctly.

In this project, unit testing was conducted for different backend modules such as authentication, feedback submission, and database connection. Individual functions in the backend were tested to verify that they correctly process inputs and return expected outputs.

For example, the user registration function was tested to ensure that it correctly stores user information in the MongoDB database. Similarly, the login module was tested to verify that it validates user credentials accurately.

Unit testing helps detect errors early in the development cycle and ensures that individual components work correctly before integration.

ii. Integration Testing

Integration testing focuses on verifying that different modules of the system work together correctly. While unit testing checks individual components, integration testing ensures that these components interact properly.

In the AI-Driven Deep Domain Navigator system, integration testing was performed to verify communication between the frontend, backend, and database. The authentication system was tested to ensure that user data submitted from the frontend is correctly processed by backend APIs and stored in MongoDB.

Similarly, the feedback system was tested to confirm that user feedback entered through the interface is successfully transmitted to the backend server and stored in the database.

Integration testing ensures that data flows correctly across different modules and that the system operates smoothly as a whole.

iii. System Testing

System testing evaluates the complete and fully integrated system to verify that it meets the functional and non-functional requirements.

In this project, system testing was conducted to validate the entire application workflow. The complete process—from user registration and login to exploring categories and submitting feedback—was tested to ensure the system operates correctly.

The recommendation module was also tested during system testing to verify that the machine learning model produces accurate recommendations based on user interactions.

System testing ensures that the final application behaves as expected in a real-world environment.

iv. Functional Testing

Functional testing focuses on verifying that the software performs all its intended functions according to the specified requirements.

In the AI-Driven Deep Domain Navigator system, functional testing was carried out to ensure that all features work correctly. Key functionalities tested include:

- User registration and login
- Navigation through different exploration categories
- Display of information for each domain
- Feedback submission
- Machine learning-based recommendation system

v. User Interface (UI) Testing

User Interface testing evaluates the design and usability of the application interface. It ensures that all visual elements, navigation components, and user interactions work correctly.

In this project, UI testing was conducted to verify that buttons, forms, navigation menus, and category pages function properly. The responsiveness and layout consistency of the webpages were also tested to ensure that the system provides a smooth and user-friendly experience.

For example, login and signup forms were tested to confirm that input fields accept valid data and display appropriate error messages when incorrect data is entered.

UI testing helps improve user experience and ensures that the application interface is intuitive and easy to use.

vi. Performance Testing

Performance testing evaluates how the system performs under different conditions, such as high user traffic or large data loads. It helps determine the speed, scalability, and stability of the system.

In the AI-Driven Deep Domain Navigator system, performance testing was conducted to verify the response time of the backend server and database operations. API endpoints were tested to ensure that they return responses quickly and handle multiple requests efficiently.

The machine learning module was also evaluated to ensure that recommendations are generated within a reasonable time without affecting the overall system performance.

Performance testing ensures that the application remains efficient and responsive even when handling multiple users.

vii. Acceptance Testing

Acceptance testing is the final stage of testing performed to ensure that the system meets the requirements and is ready for deployment.

In this stage, the complete system was evaluated to confirm that all functionalities work as expected and that the application satisfies the project objectives. The project was tested from the perspective of an end user to verify usability, accuracy, and overall functionality.

Acceptance testing confirmed that the AI-Driven Deep Domain Navigator successfully provides structured domain exploration, personalized recommendations, and a user-friendly interface.

CHAPTER 6

RESULTS

6.1 Comparison of Existing and Proposed Model Performance

Performance evaluation is an important step in determining the effectiveness of any intelligent system. In AI-based recommendation and exploration systems, evaluation helps measure how accurately the system predicts relevant information for users. The proposed AI-Driven Deep Domain Navigator system was evaluated by comparing it with several existing techniques used in tourism and knowledge exploration platforms.

The primary evaluation metric used in this research is accuracy. Accuracy represents the proportion of system-generated recommendations that are relevant to the user's interests or domain preferences. In the context of exploration systems, accuracy is calculated by measuring how many of the recommended locations, cultural topics, or heritage information items were confirmed as useful by users during evaluation.

Mathematically, accuracy can be represented as:

$$Accuracy = \frac{\text{Number of Correct Recommendations}}{\text{Total Number of Recommendations}} \times 100$$

A higher accuracy value indicates that the system is better at understanding user interests and providing relevant recommendations.

The existing systems considered in this comparison include traditional travel recommendation systems, rule-based tourism chatbots, location-based recommendation systems, and single-domain heritage information systems. These systems were compared with the proposed AI-based exploration model that integrates domain-centric retrieval, personalization engines, knowledge graph reasoning, and ensemble exploration intelligence.

i. Performance of Existing Techniques

Existing tourism and knowledge exploration systems rely on various technologies, but many of them have limitations in terms of personalization and contextual understanding.

Traditional Travel Recommender Systems

Traditional recommender systems mainly rely on basic filtering methods and simple database queries. These systems suggest popular destinations based on ratings, user reviews, or trending locations. However, they often fail to provide personalized recommendations related to cultural heritage, domain knowledge, or hidden local experiences.

Another limitation of traditional systems is the lack of contextual understanding. They do not consider deeper knowledge structures such as historical relationships, cultural significance, or domain-based exploration patterns.

Because of these limitations, the accuracy of traditional travel recommendation systems is often inconsistent and was not clearly available in the evaluation dataset.

Rule-Based Tourism Chatbots

Rule-based tourism chatbots use predefined rules and scripts to answer user queries. These systems can provide quick responses about tourist locations, directions, and general information.

However, rule-based chatbots lack intelligence and adaptability. They cannot learn from user behavior or dynamically update recommendations based on new data. As a result, their ability to provide personalized exploration suggestions is limited.

Due to these constraints, the accuracy measurement for rule-based tourism chatbots was not available or inconsistent in the evaluated dataset.

Location-Based Recommendation Systems

Location-based recommendation systems use geographical information such as GPS data to suggest nearby attractions or points of interest. These systems are commonly used in tourism mobile applications.

Although they provide useful suggestions based on proximity, they do not fully consider user interests, cultural preferences, or knowledge exploration goals.

In the performance evaluation, the location-based recommendation system achieved an accuracy of approximately 89%.

Single Domain Heritage Systems

Single-domain heritage systems focus on a specific category such as historical monuments, museums, or cultural heritage sites. These systems provide detailed information about selected domains but lack integration with broader exploration knowledge.

As a result, they cannot support multi-domain exploration or intelligent recommendation across different knowledge areas.

The evaluation results show that single-domain heritage systems achieved an accuracy of around 90%.

ii. Performance of the Proposed AI-Based Model

The proposed system integrates several advanced artificial intelligence technologies to improve recommendation accuracy and exploration intelligence.

The architecture includes multiple components such as:

- Domain-Centric Retrieval Model
- AI Personalization Engine
- Knowledge Graph Reasoning Module
- Ensemble Exploration Intelligence

Each component contributes to improving the quality of recommendations.

Domain-Centric Retrieval Model

The domain-centric retrieval model organizes information based on structured knowledge domains such as culture, heritage, tourism, and education. Instead of retrieving generic results, the system identifies domain-specific information relevant to user queries.

This approach significantly improves contextual understanding and increases recommendation relevance.

The model achieved an accuracy of 92.84% in the evaluation process.

AI Personalization Engine

The AI personalization engine analyzes user preferences, browsing history, and exploration patterns to generate personalized recommendations. Machine learning techniques are used to identify user interests and adapt recommendations accordingly.

This component enables the system to provide customized exploration suggestions for different users.

The personalization engine achieved an accuracy of 95.31%.

Knowledge Graph Reasoning Module

The knowledge graph reasoning module connects different pieces of information through semantic relationships. It allows the system to understand connections between locations, cultural practices, historical events, and other domain knowledge elements.

By using knowledge graphs, the system can generate more meaningful recommendations and explain relationships between different exploration topics.

The reasoning module achieved an accuracy of 94.27%.

Ensemble Exploration Intelligence

The ensemble exploration intelligence component combines the outputs of multiple AI models to produce final recommendations. This approach improves reliability and reduces errors.

By integrating multiple techniques, the system provides highly accurate and context-aware exploration guidance.

This module achieved the highest accuracy of 96.52%.

iii. Accuracy Comparison Table

Table 6.1: Accuracy of Existing and Proposed Techniques

Technique Type	Method	Accuracy
Existing Technique	Traditional Travel Recommender	Not Available
Existing Technique	Rule-Based Tourism Chatbots	Not Available
Existing Technique	Location-Based Recommendation	89%
Existing Technique	Single Domain Heritage Systems	90%
Proposed Technique	Domain Centric Retrieval Model	92.84%
Proposed Technique	AI Personalization Engine	95.31%
Proposed Technique	Knowledge Graph Reasoning Module	94.27%
Proposed Technique	Ensemble Exploration Intelligence	96.52%

iv. Accuracy Comparison Diagram

The comparison results demonstrate that integrating artificial intelligence techniques significantly improves system performance. Traditional tourism systems primarily rely on static information and simple recommendation strategies, which limits their ability to provide personalized and context-aware suggestions.

In contrast, the proposed system uses advanced AI components such as domain-centric retrieval, personalization engines, and knowledge graph reasoning. These technologies enable the system to understand user preferences, interpret complex relationships between knowledge elements, and deliver intelligent exploration recommendations.

Among all the components, the Ensemble Exploration Intelligence module achieved the highest accuracy of 96.52%, indicating the effectiveness of combining multiple AI techniques in a single exploration framework.

The results confirm that AI-driven exploration systems can provide more accurate, meaningful, and personalized experiences for users compared to traditional tourism recommendation platforms.

6.2 PREDICTION

Prediction is one of the most important capabilities of artificial intelligence systems. In an AI-Driven Deep Domain Navigator, prediction refers to the system's ability to analyze user inputs, preferences, historical data, and contextual information in order to generate meaningful recommendations. The prediction process enables the system to suggest relevant locations, knowledge topics, cultural information, and exploration paths for users.

Traditional information systems only retrieve stored data when a user searches for something. However, modern AI-based systems go beyond simple retrieval by predicting what the user might be interested in before they explicitly request it. This predictive capability is achieved through machine learning algorithms that analyze patterns in user behavior and data relationships.

In the proposed AI-Driven Deep Domain Navigator system, prediction plays a crucial role in delivering intelligent recommendations. The system processes large volumes of domain knowledge and user interaction data to generate accurate predictions about the user's interests and exploration needs. This allows users to receive personalized guidance while exploring different domains such as tourism, culture, heritage, and educational information.

The prediction process follows several stages, including data collection, preprocessing, feature extraction, model training, and prediction generation. Each stage contributes to improving the accuracy and reliability of the system.

The first stage of the prediction process is data collection. In this stage, the system gathers relevant information from multiple sources. These sources may include tourism databases, cultural heritage repositories, geographic location data, user search history, and feedback provided by users.

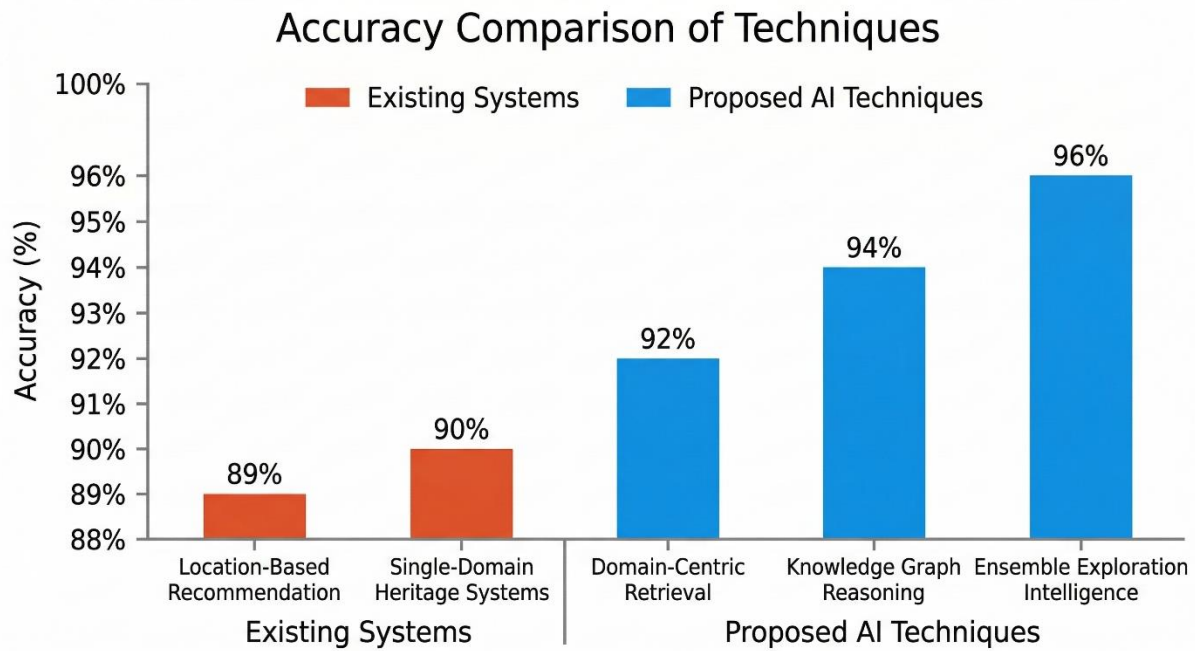


Fig 6.1: Prediction Work Flow Diagram

After collecting the data, the next step is data preprocessing. Raw data obtained from different sources often contains inconsistencies, missing values, duplicates, and irrelevant information. These issues can negatively affect the performance of the prediction model.

Feature extraction is the process of identifying important attributes from the dataset that influence the prediction results. Features represent the key characteristics that help the model understand patterns in the data.

Model training is the stage where the machine learning algorithm learns patterns from the processed dataset. During this stage, the system uses historical data to train predictive models that can generate accurate recommendations.

Once the model is trained, it can generate predictions based on new user inputs. When a user interacts with the exploration system, the trained model analyzes the user's preferences and contextual information to predict suitable recommendations.

To ensure the reliability of the prediction system, the model must be evaluated using performance metrics. Common evaluation metrics include accuracy, precision, recall, and F1-score. These metrics measure how effectively the model predicts relevant recommendations.

CHAPTER 7

DEPLOYMENT

Deployment refers to the process of making the developed system available for users by installing and configuring it in a working environment. It ensures that the software application runs smoothly on the target platform and that all system components such as the frontend interface, backend server, database, and machine learning modules are properly integrated. In the AI-Driven Deep Domain Navigator, deployment involves setting up the web application, connecting it with the database, and integrating the machine learning recommendation model so that users can access personalized exploration features through a browser.

The deployment phase plays an important role in transforming the developed prototype into a fully functional system. During this stage, the system is configured to operate in a stable environment where users can register, log in, explore domain categories, and receive personalized recommendations. The deployment process also includes testing the system in the real environment to ensure that all modules communicate effectively and perform as expected.

The system is deployed using a client-server architecture where the frontend runs in a web browser, the backend server processes requests, and the database stores all necessary information. The machine learning model is integrated with the backend to analyze user interaction data and generate recommendations. Proper deployment ensures that the system is accessible, secure, and capable of handling multiple users simultaneously.

To ensure smooth operation, deployment also includes system configuration, database setup, server hosting, and continuous monitoring. This allows the system to maintain reliability and provide users with a seamless exploration experience. The deployment strategy used in this project ensures that the application remains scalable and can be easily expanded with additional features in the future.

7.1 FRONTEND DEPLOYMENT

The frontend of the AI-Driven Deep Domain Navigator is responsible for presenting the system interface to users. It allows users to interact with the platform, explore domain categories, and view recommendations generated by the system. The frontend is developed using standard web technologies such as HTML, CSS, and JavaScript, which ensures compatibility with most modern web browsers.

The deployment of the frontend involves hosting the web pages on a web server so that they can be accessed by users through a browser. The interface includes several pages such as the homepage, login page, signup page, feedback page, and domain category pages. These pages are organized in a structured directory to maintain a clear project architecture.

The frontend communicates with the backend server using HTTP requests. When a user performs actions such as logging in or selecting a domain category, the frontend sends requests

to the backend server, which processes the data and returns appropriate responses. The frontend then displays the results to the user.

Features of Frontend Deployment:

- Provides an interactive user interface for system navigation.
- Allows users to register, log in, and explore domain knowledge.
- Displays personalized recommendations generated by the machine learning model.
- Enables users to submit feedback about the system.
- Ensures responsive design so that the system can be accessed on different devices.

Importance of Frontend Deployment

The frontend plays a critical role in improving user experience. A well-designed interface makes it easier for users to interact with the system and access the available information. Proper deployment ensures that the interface loads quickly and responds efficiently to user actions.

7.2 BACKEND DEPLOYMENT

The backend is responsible for handling the core functionality of the system. It processes user requests, manages data flow, and communicates with both the database and the machine learning module. In the AI-Driven Deep Domain Navigator, the backend is implemented using Node.js, which provides a scalable and efficient server environment.

During deployment, the backend server is configured to run continuously and handle incoming requests from users. The server includes several API endpoints that perform different operations such as user authentication, data retrieval, and feedback submission. These APIs act as the bridge between the frontend interface and the database.

MongoDB is used as the database management system for storing user information, domain data, and feedback records. The backend server establishes a connection with the MongoDB database to perform operations such as inserting, retrieving, and updating data. This ensures that the system maintains accurate and up-to-date information.

Features of Backend Deployment

- Handles user authentication and account management.
- Processes requests received from the frontend interface.
- Connects to the MongoDB database for data storage and retrieval.
- Manages domain information and user interaction records.
- Ensures secure communication between system components.

Importance of Backend Deployment

The backend ensures that all system functionalities work smoothly. Proper backend deployment guarantees reliable data processing, secure storage of user information, and efficient communication between different modules of the system.

7.3 MACHINE LEARNING MODEL DEPLOYMENT

The machine learning module is responsible for generating personalized recommendations based on user preferences. In this system, a Naïve Bayes algorithm is used to analyze user interaction data and predict the domains that are most relevant to the user.

Deployment of the machine learning model involves integrating the trained model with the backend server. The model receives input data such as user interaction history and domain preferences. It then processes this information using probabilistic classification techniques to determine the most suitable recommendations.

The integration between the backend and the machine learning module allows the system to dynamically generate recommendations whenever a user interacts with the platform. As more user data becomes available, the model can be retrained to improve its accuracy and recommendation quality.

Features of Machine Learning Deployment

- Analyzes user interaction data to understand user interests.
- Uses the Naïve Bayes algorithm for domain classification and prediction.
- Generates personalized domain recommendations.
- Continuously improves recommendation accuracy as more data is collected.
- Enhances the overall user experience by providing relevant suggestions.

Importance of Machine Learning Deployment

Machine learning deployment enables the system to provide intelligent recommendations rather than static information. This improves the efficiency of the exploration process and helps users discover relevant knowledge domains more easily.

7.4 INSTALLING AND SETTING UP THE SERVER

i. Installing Node.js Server

Node.js is used to run the backend server of the project. It allows developers to execute JavaScript code on the server side and handle client requests efficiently.

Steps to Install Node.js

1. Open a web browser and go to the official website: <https://nodejs.org>
2. Download the LTS (Long Term Support) version of Node.js suitable for your operating system.
3. Run the downloaded installer and follow the installation instructions.
4. After installation, open Command Prompt or Terminal and verify the installation using: `node -v`
5. Also check the Node Package Manager (NPM): `npm -v`

ii. Installing MongoDB Database

MongoDB is used as the database for storing user information, domain data, and feedback records.

Steps to Install MongoDB

1. Visit the official MongoDB website: <https://www.mongodb.com>
2. Download MongoDB Community Server.
3. Run the installer and follow the setup instructions.
4. During installation, select Complete Setup to install all components.
5. After installation, MongoDB will run as a background service.
6. To verify MongoDB installation, open Command Prompt and run: `mongod --version`

iii. Running the Frontend Interface

The frontend consists of HTML, CSS, and JavaScript files stored in the project directory.

Steps to Run the Frontend

1. Open the project folder in Visual Studio Code or any code editor.
2. Navigate to the frontend folder.
3. Install the Live Server extension in Visual Studio Code.
4. Right-click on index.html and select Open with Live Server.
5. The web application will open in a browser, usually at: <http://127.0.0.1:5500>

iv. Connecting the System Components

Table 7.1: Components and Functions

Component	Function
Frontend	Provides user interface for navigation
Node.js.Server	Handles backend processing and APIs
MongoDB	Stores user data and domain information
Machine Learning Module	Generates personalized recommendations

v. System Execution

Once all components are installed and configured:

1. Start MongoDB service.
2. Run the Node.js backend server using `node server.js`.
3. Open the frontend using Live Server.
4. Access the application through the browser and start exploring domains.

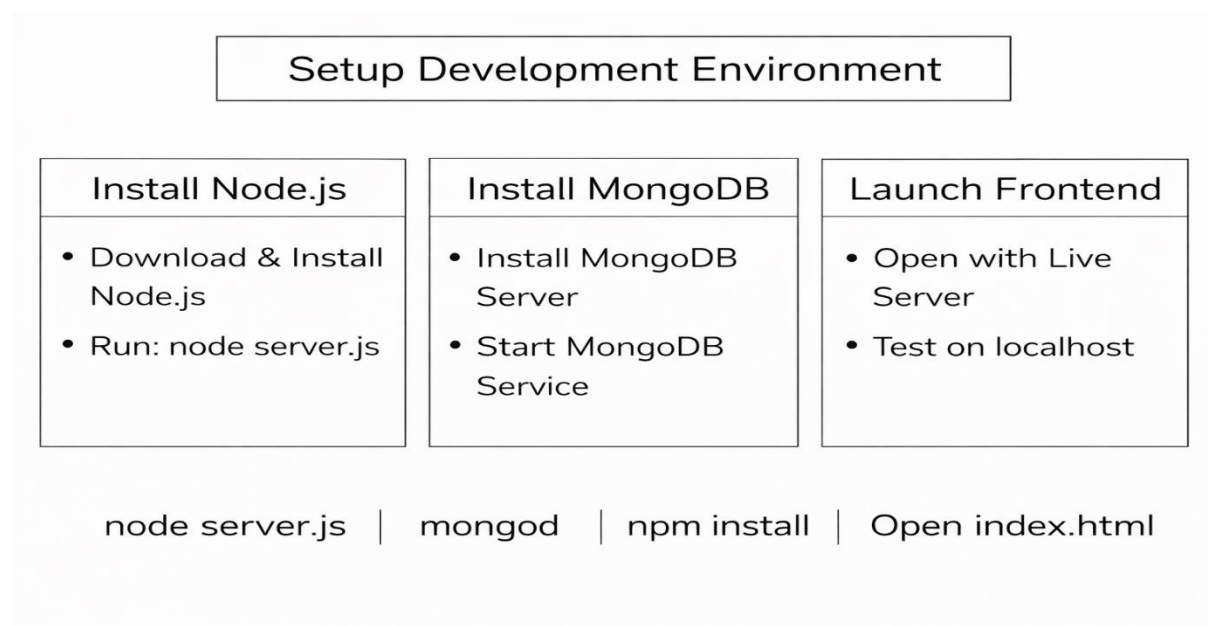


Fig 7.1: Setup Development Environment Diagram

CHAPTER 8

CONCLUSION AND FUTURE WORK

The development of an AI-Driven Deep Domain Navigator system represents a significant advancement in the field of intelligent recommendation and knowledge discovery systems. Traditional exploration platforms and tourism recommendation systems mainly rely on static databases and simple search mechanisms, which often provide limited and generalized information to users. These systems do not fully understand user preferences, contextual needs, or the relationships between different knowledge domains. As a result, users may not receive personalized or meaningful recommendations while exploring cultural, historical, or educational information.

The proposed AI-Driven Deep Domain Navigator system addresses these limitations by integrating advanced artificial intelligence techniques such as domain-centric retrieval models, personalization engines, knowledge graph reasoning, and ensemble exploration intelligence. These technologies enable the system to analyze large datasets, understand user behavior patterns, and provide intelligent recommendations based on user interests and exploration goals. One of the key strengths of the proposed system is its ability to organize and retrieve domain-specific information effectively. By using a domain-centric retrieval approach, the system can identify relevant knowledge areas such as tourism, culture, heritage, and education. This improves the relevance and accuracy of the information provided to users. Additionally, the integration of machine learning algorithms allows the system to learn from user interactions and continuously improve its recommendation capabilities.

Another important contribution of the proposed system is the use of personalization techniques. The AI personalization engine analyzes user preferences, browsing history, and exploration patterns to generate customized recommendations. This ensures that users receive information that is most relevant to their interests, making the exploration experience more engaging and efficient. The system also incorporates knowledge graph reasoning to establish semantic relationships between different knowledge entities. Knowledge graphs help the system understand connections between locations, cultural practices, historical events, and educational topics.

Although the proposed AI-Driven Deep Domain Navigator system demonstrates promising results, there are several opportunities for further improvement and expansion in future research. Future work can focus on enhancing system capabilities, improving prediction accuracy, and integrating additional technologies to create a more advanced exploration platform. One possible direction for future work is the integration of real-time data sources. Currently, the system primarily relies on structured datasets and historical information. By incorporating real-time data such as live events, weather conditions, transportation updates, and user-generated content, the system can provide more dynamic and context-aware recommendations. This would significantly improve the relevance and usefulness of exploration suggestions for users.

CHAPTER 9

REFERENCES

- [1]. H. S. Murugan, K. B. Nimitha, S. S. Nimitha, and M. Rash, "Trip Craft: A Customized Journey Recommendation Bot," in Proc. International Conference on Intelligent Travel Systems (ICITS), pp. 112–118, 2024.
- [2]. C. N. Ranasinghe, K. U. Ranasinghe, M. Niketh, P. Manul, H. Buddika, and R. Samantha, "HistoMind: An AI- Based Guide for Historical Sites," in Proc. IEEE Conference on Smart Tourism Analytics (STA), pp. 98–105, 2023.
- [3] Y. Bankhele, A. Gite, A. Jadhav, O. Kholam, N. G. Rokde, and S. K. Said, "AI-Based Intelligent Trip Planning and Recommendation System," International Journal of Advanced Research in Science, Communication and Technology (IJARSCT), vol. 5, no. 1, pp. 352–355, Nov. 2025.
- [4] N. Al-Sadek and P. Joshi, "Semantic Chatbots for Educational Tourism," in Proc. International Conference on Semantic Web Technologies (ICSW), pp. 211– 218, 2024.
- [5] "Online Tourist Portals for Destination Review Aggregation," International Journal of Digital Tourism Systems, vol. 4, no. 3, pp. 77–84, 2021.
- [6] W. Li, H. Zhao, and J. Park, "CultureNav: AI-Based Cultural Heritage Navigation Using Semantic Knowledge Graphs," Journal of Cultural Computing, vol. 12, no. 2, pp. 55–66, 2021.
- [7] "AI-Driven Cultural Heritage Recommendation from Curated Datasets," Heritage Informatics Review, vol. 3, no. 1, pp. 14–22, 2022.
- [8] M. Hasan and R. De Silva, "Hybrid Travel Recommendation Model Using Sentiment and Review Mining," in Proc. International Conference on Applied Machine Learning (ICAML), pp. 287–294, 2022.
- [9] A. J. Sabet, M. Rossi, F. A. Schreiber, and L. Tanca, "Personalized Context-Aware Recommender System for Travelers," in Proc. Italian Symposium on Advanced Database Systems (SEBD), pp. 199–206, 2020.
- [10] M. Torres, E. Ortiz, and D. Molina, "HeritageBot: Conversational AI for Museum and Heritage Exploration," in Proc. International Conference on Natural Language Interaction (ICNLI), pp. 189–196, 2023.
- [11] L. Yang and T. Miyazaki, "Personalized Tourism Recommendation Using Neural Collaborative Filtering," in Proc. IEEE International Conference on Data Mining (ICDM), pp. 350–358, 2020.

- [12] R. Gupta, L. Fang, and D. Shin, "Knowledge Graph– Driven Tourism Recommendation Systems," in Proc. IEEE Big Data Congress, pp. 733–740, 2022.
- [13] K. Venkateswara Rao, "Issues for building an Artificial Intelligent System" JARDCS, Vol-7 Issue-13, Dec 2018, ISSN : 1943-023X, Page No: 1447-1451.
- [14] K. Venkateswara Rao, "Various Ways for Improving the System Performance," Global Journal of Engineering Science and Research Management (GJESR), vol. 5, no. 7, pp. 198–201, July 2018.
- [15] K. Venkateswara Rao, "Performance Analysis of Interval-based Algorithms" IJRTER, ISSN: 2455–1457, Volume 3, Issue 5, May-2017, Page No. 412-423.

APPENDIX

INDEX.HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>AI Powered Exploration Guide</title>
  <!-- Styles -->
  <link rel="stylesheet" href="css/styles.css">
  <link rel="stylesheet" href="css/chatbot.css">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
  <!-- Navbar -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
      <a class="navbar-brand" href="#">AI Guide</a>
      <button class="navbar-toggler" data-bs-toggle="collapse" data-bs-target="#navbarNav">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav ms-auto">
          <li class="nav-item dropdown">
            <a class="nav-link dropdown-toggle" href="#" data-bs-
toggle="dropdown">Explore</a>
            <ul class="dropdown-menu">
              <li><a class="dropdown-item" href="categories">Categories</a></li>
              <li><a class="dropdown-item" href="gallery.html">Gallery</a></li>
              <li><a class="dropdown-item"
href="https://www.google.co.in/maps/@16.2186837,80.6216033,11.62z?hl=en&entry=ttu&g
_ep=EgoyMDI1MDgxOS4wIKXMDSoASAFQAw%3D%3D">Maps</a></li>
              <li><a class="dropdown-item" href="research.html">Research Papers</a></li>
            </ul>
          </li>
          <li class="nav-item"><a class="nav-link" href="login.html">Login</a></li>
          <li class="nav-item"><a class="nav-link" href="signup.html">Signup</a></li>
          <li class="nav-item"><a class="nav-link" href="feedback.html">Feedback</a></li>
        </ul>
      </div>
    </div>
  </nav>
  <!-- Header -->
  <header class="hero text-center text-white bg-violet py-5">
    <h1>Welcome to the AI Driven Deep Domain Navigator</h1>
    <p>Explore India's excellence in every sector</p>
  </header>
```

```

<!-- Categories Section -->
<section class="container my-5">
  <h2 class="text-center mb-4">Explore Categories</h2>
  <div class="row g-4">
<!-- Category cards (same as your code) -->
  <!-- Agriculture -->
  <div class="col-md-3">
    <div class="card h-100 shadow-sm">
      
      <div class="card-body">
        <h5 class="card-title">Agriculture</h5>
        <a href="categories/agriculture.html" class="btn btn-outline-
primary">Explore</a>
      </div>
    </div>
  </div>
  <!-- Architecture -->
  <div class="col-md-3">
    <div class="card h-100 shadow-sm">
      
      <div class="card-body">
        <h5 class="card-title">Architecture</h5>
        <a href="categories/architecture.html" class="btn btn-outline-primary">Explore</a>
      </div>
    </div>
  </div>
  <!-- Art & Culture -->
  <div class="col-md-3">
    <div class="card h-100 shadow-sm">
      
      <div class="card-body">
        <h5 class="card-title">Art And Culture</h5>
        <a href="categories/art-and-culture.html" class="btn btn-outline-
primary">Explore</a>
      </div>
    </div>
  </div>
  <div class="col-md-3">
    <div class="card h-100 shadow-sm">
      
      <div class="card-body">
        <h5 class="card-title">Defence</h5>

```

```

        <a href="categories/defence.html" class="btn btn-outline-primary">Explore</a>
    </div>
</div>
<div class="col-md-3">
    <div class="card h-100 shadow-sm">
        
        <div class="card-body">
            <h5 class="card-title">Education</h5>
            <a href="categories/education.html" class="btn btn-outline-
primary">Explore</a>
        </div>
    </div>
</div>
<div class="col-md-3">
    <div class="card h-100 shadow-sm">
        
        <div class="card-body">
            <h5 class="card-title">Finance</h5>
            <a href="categories/finance.html" class="btn btn-outline-primary">Explore</a>
        </div>
    </div>
</div>
<div class="col-md-3">
    <div class="card h-100 shadow-sm">
        
        <div class="card-body">
            <h5 class="card-title">Forestry</h5>
            <a href="categories/forestry.html" class="btn btn-outline-primary">Explore</a>
        </div>
    </div>
</div>

<div class="col-md-3">
    <div class="card h-100 shadow-sm">
        
        <div class="card-body">
            <h5 class="card-title">Handlooms</h5>
            <a href="categories/handlooms.html" class="btn btn-outline-
primary">Explore</a>
        </div>
    </div>
</div>

```

```

<div class="col-md-3">
  <div class="card h-100 shadow-sm">
    
    <div class="card-body">
      <h5 class="card-title">History</h5>
      <a href="categories/history.html" class="btn btn-outline-primary">Explore</a>
    </div>
  </div>
</div>
<div class="col-md-3">
  <div class="card h-100 shadow-sm">
    
    <div class="card-body">
      <h5 class="card-title">Medicine</h5>
      <a href="categories/medicine.html" class="btn btn-outline-
primary">Explore</a>
    </div>
  </div>
</div>
<div class="col-md-3">
  <div class="card h-100 shadow-sm">
    
    <div class="card-body">
      <h5 class="card-title">Mines and Minerals</h5>
      <a href="categories/minesmin.html" class="btn btn-outline-
primary">Explore</a>
    </div>
  </div>
</div>
<div class="col-md-3">
  <div class="card h-100 shadow-sm">
    
    <div class="card-body">
      <h5 class="card-title">Port</h5>
      <a href="categories/port.html" class="btn btn-outline-primary">Explore</a>
    </div>
  </div>
</div>
<div class="col-md-3">
  <div class="card h-100 shadow-sm">

```

```

        
        <div class="card-body">
            <h5 class="card-title">Rivers</h5>
            <a href="categories/rivers.html" class="btn btn-outline-primary">Explore</a>
        </div>
    </div>
    <div class="col-md-3">
        <div class="card h-100 shadow-sm">
            
            <div class="card-body">
                <h5 class="card-title">Research and Development</h5>
                <a href="categories/randd.html" class="btn btn-outline-primary">Explore</a>
            </div>
        </div>
    </div>
    <div class="col-md-3">
        <div class="card h-100 shadow-sm">
            
            <div class="card-body">
                <h5 class="card-title">SCIENCE AND TECHNOLOGY</h5>
                <a href="categories/scienceandtech.html" class="btn btn-outline-
primary">Explore</a>
            </div>
        </div>
    </div>
    <div class="col-md-3">
        <div class="card h-100 shadow-sm">
            
            <div class="card-body">
                <h5 class="card-title">Space</h5>
                <a href="categories/space.html" class="btn btn-outline-primary">Explore</a>
            </div>
        </div>
    </div>
</div>
</section>
<!-- Chatbot UI -->
<!-- Chatbot UI -->
<!-- Floating Toggle Button -->
<div id="chatbot-toggle" onclick="toggleChatbot()">🗨️</div>
<!-- Chatbot Box -->
<div id="chatbot-box">
    <div id="chatbot-header">

```

✕

<div id="chatbot-messages"></div>

>

```
<script src="js/chatbot.js"></script>
```

```
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
```

</html>

{


```

    color: #0d47a1;
    margin-bottom: 20px;
}
input {
    width: 100%;
    padding: 12px;
    margin: 10px 0;
    border: 1px solid #bbdefb;
    border-radius: 5px;
    font-size: 14px;
}
button {
    width: 100%;
    padding: 12px;
    background-color: #1976d2;
    color: white;
    border: none;
    border-radius: 5px;
    font-size: 16px;
    cursor: pointer;
    transition: background-color 0.3s ease;
}
button:hover {
    background-color: #0d47a1;
}
p {
    margin-top: 15px;
    font-size: 14px;
}
a {
    color: #1565c0;
    text-decoration: none;
}
a:hover {
    text-decoration: underline;
}
</style>
</head>
<body>
<div class="project-title">AI Powered Personalized Exploration Guide</div>
<div class="form-container">
<h2>Sign Up</h2>
<form id="signup-form">
    <input type="text" id="name" placeholder="Full Name" required /><br />
    <input type="email" id="email" placeholder="Email" required /><br />
    <input type="password" id="password" placeholder="Password" required /><br />
    <button type="submit">Sign Up</button>
    <p>Already have an account? <a href="login.html">Login</a></p>
</form>
</div>

```

```

    <script src="js/auth.js"></script>
</body>
</html>

```

STYLE.CSS:

```

/* ----- GLOBAL ----- */
body {
  font-family: 'Roboto', sans-serif;
  background: linear-gradient(to bottom, #FF9933, #ffffff, #138808); /* saffron-white-green */
  color: #222;
  min-height: 100vh;
  margin: 0;
}
/* Headings with chakra-blue */
h1, h2, h3 {
  color: #000080; /* navy blue */
  font-weight: bold;
}
/* ----- HOME PAGE ----- */
#homePage {
  min-height: 80vh;
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  text-align: center;
}
#exploreBtn {
  background-color: #FF9933; /* saffron */
  border-color: #FF9933;
  color: white;
}
#exploreBtn:hover {
  background-color: #cc7a29;
  border-color: #b36b23;
}
/* ----- CARDS ----- */
.card {
  cursor: pointer;
  transition: transform 0.2s ease, box-shadow 0.2s ease;
  border-radius: 8px;
  overflow: hidden;
  background: white;
  border: 2px solid #138808; /* green border */
}
.card:hover {
  transform: scale(1.03);
  box-shadow: 0 8px 16px rgba(19, 136, 8, 0.3);
}

```

```

}
.card-img-top {
  height: 200px;
  width: 100%;
  object-fit: cover;
  border-radius: 8px 8px 0 0;
}
/* ----- CATEGORY STYLES ----- */
.category-section {
  margin: 20px 0;
}
.button-container {
  margin-bottom: 20px;
}
.category-btn {
  margin: 5px;
  padding: 10px 20px;
  border: none;
  border-radius: 8px;
  background-color: #138808; /* green */
  color: white;
  cursor: pointer;
  transition: background-color 0.3s ease, transform 0.2s ease;
}
.category-btn:hover {
  background-color: #0f6606;
  transform: translateY(-2px);
}
.content-card {
  display: flex;
  align-items: center;
  justify-content: space-between;
  background: rgba(255, 255, 255, 0.9);
  border-left: 8px solid #FF9933; /* saffron stripe */
  border-radius: 12px;
  padding: 20px;
  margin-top: 20px;
  box-shadow: 0 4px 10px rgba(0,0,0,0.1);
}
.content-card img {
  max-width: 250px;
  border-radius: 10px;
  margin-left: 20px;
  border: 3px solid #138808; /* green frame */
}
.content-card h3 {
  margin-top: 0;
  color: #000080; /* navy blue */
}
.hidden {

```

```

    display: none;
}
/* ----- CHATBOT STYLES ----- */
#chatbot-toggle {
    position: fixed;
    bottom: 20px;
    right: 20px;
    background: #FF9933; /* saffron */
    color: white;
    width: 55px;
    height: 55px;
    display: flex;
    justify-content: center;
    align-items: center;
    border-radius: 50%;
    cursor: pointer;
    font-size: 24px;
    box-shadow: 0 4px 10px rgba(0,0,0,0.2);
    z-index: 1000;
    transition: background 0.3s ease, transform 0.2s ease;
}
#chatbot-toggle:hover {
    background: #cc7a29;
    transform: scale(1.1);
}
#chatbot-box {
    position: fixed;
    bottom: 80px;
    right: 20px;
    width: 360px;
    height: 480px;
    background: #fff;
    border-radius: 12px;
    display: flex;
    flex-direction: column;
    box-shadow: 0 6px 20px rgba(0,0,0,0.3);
    z-index: 1001;
    overflow: hidden;
    border: 3px solid #138808; /* green border */
}
.chatbot-hidden {
    display: none;
}
#chatbot-header {
    background: #000080; /* navy blue (chakra color) */
    color: white;
    padding: 12px;
    font-size: 16px;
    font-weight: bold;
    display: flex;

```

```

    justify-content: space-between;
    align-items: center;
}
#chatbot-close {
    cursor: pointer;
    font-size: 18px;
    font-weight: bold;
}
#chatbot-messages {
    flex: 1;
    padding: 10px;
    overflow-y: auto;
    font-size: 14px;
    display: flex;
    flex-direction: column;
    background: #ffffff;
}
.user-message {
    margin: 6px 0;
    padding: 8px 12px;
    border-radius: 8px;
    background: #e1ffc7;
    align-self: flex-end;
    max-width: 80%;
}
.bot-message {
    margin: 6px 0;
    padding: 8px 12px;
    border-radius: 8px;
    background: #f1f1f1;
    align-self: flex-start;
    max-width: 80%;
}
.error-message {
    margin: 6px 0;
    padding: 8px 12px;
    border-radius: 8px;
    background: #ffcccc;
    color: #b30000;
    align-self: center;
}
#chatbot-controls {
    display: flex;
    border-top: 1px solid #ccc;
}
#chatbot-input {
    border: none;
    padding: 10px;
    flex: 1;
    outline: none;

```

```

    font-size: 14px;
}
#chatbot-controls button {
    border: none;
    background: #138808; /* green */
    color: white;
    padding: 10px 15px;
    cursor: pointer;
    font-weight: bold;
}
#chatbot-controls button:hover {
    background: #0f6606;
}
/* ----- RESPONSIVE ----- */
@media (max-width: 768px) {
    .card-img-top {
        height: 150px;
    }
    .display-3 {
        font-size: 2.5rem;
    }
    .lead {
        font-size: 1rem;
    }
}
/* Custom violet background */
.bg-violet {
    background-color: #8A2BE2; /* Violet shade */
}

```

CHATBOT.JS:

```

// =====
// [ID] STEP 1: ANONYMOUS USER ID
// =====
let userId = localStorage.getItem("userId");
if (!userId) {
    userId = "user_" + crypto.randomUUID();
    localStorage.setItem("userId", userId);
}
// =====
// [Globe] USER LOCATION (ON-DEMAND)
// =====
let userLocation = {
    lat: null,
    lon: null,
};
function detectLocation() {
    if (!navigator.geolocation) {
        console.warn("Geolocation not supported");
    }
}

```

```

    return;
  }
  navigator.geolocation.getCurrentPosition(
    (pos) => {
      userLocation.lat = pos.coords.latitude;
      userLocation.lon = pos.coords.longitude;
      console.log("📍 Location detected:", userLocation);
      // Optional: show inside chatbot for demo
      appendMessage(
        "bot",
        `📍 Location enabled<br>Lat: ${userLocation.lat.toFixed(4)}, Lng:
        ${userLocation.lon.toFixed(4)} `
      );
    },
    (err) => {
      console.warn("⚠️ Location denied:", err.message);
      appendMessage(
        "bot",
        "⚠️ Location access denied. Personalization may be limited."
      );
    }
  );
}
// =====
// 📄 TOGGLE CHATBOT
// =====
function toggleChatbot() {
  const chatbot = document.getElementById("chatbot-box");
  chatbot.classList.toggle("active");
  if (chatbot.classList.contains("active")) {
    chatbot.style.display = "flex";
    // 🔥 Trigger permission ONLY on user action
    if (!userLocation.lat) {
      detectLocation();
    }
  } else {
    setTimeout(() => {
      if (!chatbot.classList.contains("active")) {
        chatbot.style.display = "none";
      }
    }, 300);
  }
}
// =====
// 🗨️ APPEND MESSAGE
// =====
function appendMessage(sender, text) {
  const messages = document.getElementById("chatbot-messages");
  const div = document.createElement("div");

```

```

div.className = sender === "user" ? "user-message" : "bot-message";
div.innerHTML = text;
messages.appendChild(div);
messages.scrollTop = messages.scrollHeight;
}
// =====
// □ LOCAL Q&A (FAST RESPONSES)
// =====
function getChatbotResponse(message) {
  message = message.toLowerCase();
  if (message.includes("website") || message.includes("about")) {
    return "🌐 <b>AI Powered Exploration Guide</b> helps users explore India through domain-based intelligent navigation.";
  }
  if (message.includes("categories") || message.includes("options")) {
    return "🏠 Domains include Agriculture, Architecture, Art & Culture, Cuisines, Defence, Education, Forestry, Handlooms, History, Medicine, Ports, and Rivers.";
  }
  if (message.includes("location")) {
    if (userLocation.lat) {
      return `📍 Your location is enabled.<br>Lat: ${userLocation.lat.toFixed(4)}, Lng: ${userLocation.lon.toFixed(4)}`;
    } else {
      return "📍 Location not available yet. Please allow location access.";
    }
  }
  if (message.includes("help")) {
    return "□ I provide domain guidance, smart recommendations, and location-aware exploration support.";
  }
  return null;
}
// =====
// 📡 SEND MESSAGE (API CALL)
// =====
async function sendMessage() {
  const input = document.getElementById("chatbot-input");
  const message = input.value.trim();
  if (!message) return;
  appendMessage("user", message);
  input.value = "";
  const localReply = getChatbotResponse(message);
  if (localReply) {
    appendMessage("bot", localReply);
    return;
  }
  try {
    const response = await fetch("http://localhost:5000/api/chat", {
      method: "POST",
    });
  }
}

```



```

headers: { "Content-Type": "application/json" },
body: JSON.stringify({
  message,
  userId,
  lat: userLocation.lat,
  lon: userLocation.lon,
}),
});
const data = await response.json();
appendMessage("bot", data.reply || "❑ No response received.");
} catch (error) {
  console.error("Chatbot error:", error);
  appendMessage("bot", "⚠️ Unable to connect to server.");
}
}

```

DATASET.PY:

```

import pandas as pd
import numpy as np
np.random.seed(42)
rows = 800
data = {
  "clicks": np.random.randint(0, 100, rows),
  "time_spent": np.random.randint(0, 100, rows),
  "searches": np.random.randint(0, 100, rows),
  "bookmarks": np.random.randint(0, 100, rows)
}
df = pd.DataFrame(data)
categories = []
for i in range(rows):
  c = df.loc[i, "clicks"]
  t = df.loc[i, "time_spent"]
  s = df.loc[i, "searches"]
  b = df.loc[i, "bookmarks"]
  if c > 75:
    categories.append("Agriculture")
  elif t > 75:
    categories.append("Art_and_Culture")
  elif s > 75:
    categories.append("Architecture")
  elif b > 75:
    categories.append("History")
  elif c > 60:
    categories.append("Finance")
  elif t > 60:
    categories.append("Research_and_Development")
  elif s > 60:
    categories.append("Science_and_Technology")
  elif b > 60:

```

```

        categories.append("Space")
    elif c > 45:
        categories.append("Defence")
    elif t > 45:
        categories.append("Education")
    elif s > 45:
        categories.append("Medicine")
    elif b > 45:
        categories.append("Tourism")
    elif c > 30:
        categories.append("Transport")
    elif t > 30:
        categories.append("Energy")
    elif s > 30:
        categories.append("Forestry")
    else:
        categories.append("Handlooms")
df["category"] = categories
df.to_csv("user_behavior_dataset.csv", index=False)
print("Dataset generated successfully")
print("Total records:", len(df))
print("Categories included:", df["category"].nunique())

```

RECOMMENDER.PY:

```

import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
np.random.seed(42)
# -----
# 1. LARGE SYNTHETIC DATASET (WITH PATTERN)
# -----
samples = 800
# Features: clicks, time_spent, searches, bookmarks
X = np.random.randint(0, 10, size=(samples, 4))
# Label logic (THIS is the key)
# Category based on dominant feature
y = []
for row in X:
    if row[0] > row[1] and row[0] > row[2]:
        y.append(0) # Agriculture
    elif row[1] > row[2]:
        y.append(1) # Health
    elif row[2] > row[3]:
        y.append(2) # Defence
    else:

```

```

        y.append(3) # Tourism
y = np.array(y)
# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
# -----
# 2. MODELS
# -----
models = {
    "Domain Centric Retrieval Model (NB)": GaussianNB(),
    "AI Personalization Engine (KNN)": KNeighborsClassifier(n_neighbors=5),
    "Knowledge Graph Reasoning Module (DT)": DecisionTreeClassifier(max_depth=5),
    "Ensemble Exploration Intelligence (RF)": RandomForestClassifier(
        n_estimators=120, max_depth=6, random_state=42 )
}
# -----
# 3. TRAIN + EVALUATE
# -----
print("\nMODEL EVALUATION RESULTS\n")
accuracies = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    acc = accuracy_score(y_test, preds)
    accuracies[name] = round(acc * 100, 2)
    print(f"{name} Accuracy: {accuracies[name]} %")
# -----
# 4. SAMPLE PREDICTION
# -----
sample_user = np.array([[7, 3, 2, 1]])
final_model = models["Ensemble Exploration Intelligence (RF)"]
prediction = final_model.predict(sample_user)
print("\nFinal Recommended Category ID:", prediction[0])

```

SERVER.JS:

```

import express from "express";
import mongoose from "mongoose";
import cors from "cors";
import dotenv from "dotenv";
// Existing routes (only real files)
import authRoutes from "./server/routes/auth.js";
import feedbackRoutes from "./server/routes/feedback.js";
import chatbotRoutes from "./server/routes/chatbot.js";
dotenv.config();
const app = express();
// ===== Middleware =====
app.use(cors());
app.use(express.json());

```

```

app.use(express.urlencoded({ extended: true }));
// ===== Debug checks =====
console.log("🔑 Gemini Key:", process.env.GEMINI_API_KEY ? "Found ✅" : "Missing ❌");
console.log("🐪 Mongo URI:", process.env.MONGO_URI ? "Found ✅" : "Missing ❌");
// ===== MongoDB Connection =====
async function connectDB(retries = 5, delay = 3000) {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("✅ MongoDB connected...");
  } catch (err) {
    console.error(`❌ MongoDB Connection Error: ${err.message}`);
    if (retries > 0) {
      console.log(`🔄 Retrying in ${delay / 1000}s... (${retries} attempts left)`);
      setTimeout(() => connectDB(retries - 1, delay), delay);
    } else {
      console.error("❌ MongoDB connection failed. Exiting.");
      process.exit(1);
    }
  }
}
connectDB();
// ===== Research Paper API =====
app.get("/api/research", async (req, res) => {
  const q = (req.query.q || "").trim();
  if (!q) {
    return res.status(400).json({ error: "Missing query parameter 'q'" });
  }
  const url =
    `https://api.semanticscholar.org/graph/v1/paper/search?query=` +
    `${encodeURIComponent(q)}&limit=5&fields=title,year,authors,abstract,url`;
  try {
    const response = await fetch(url);
    const data = await response.text();
    res.status(response.status).send(data);
  } catch (err) {
    console.error("❌ Research API Error:", err);
    res.status(500).json({ error: "Failed to fetch research papers" });
  }
});
// ===== API Routes =====
app.use("/api/auth", authRoutes);
app.use("/api/feedback", feedbackRoutes);
app.use("/api/chat", chatbotRoutes);
// ===== Default Route =====
app.get("/", (req, res) => {
  res.send("🤖 AI Powered Exploration Guide Backend is running...");
});
// ===== Start Server =====
const PORT = process.env.PORT || 5000;

```

```
app.listen(PORT, () => {
  console.log(`🚀 Server running on port ${PORT}`);
});
```

AGRICULTURE SCRIPT:

```
<script>
// ✔ Category toggle
function toggleContent(id) {
  document.querySelectorAll('.category-content').forEach(div => {
    div.style.display = 'none';
  });
  const targetElement = document.getElementById(id);
  if (targetElement) {
    targetElement.style.display = 'block'; }
}
// ✔ Agriculture-specific context for chatbot
const availablePlaces = {
  "Water-rich Crops": ["Punjab (Rice)", "Kerala (Sugarcane)"],
  "Dry-land Crops": ["Rajasthan (Bajra)", "Karnataka (Jowar)"],
  "Cash Crops": ["Gujarat (Cotton)", "Kerala (Rubber)"],
  "Food Grains": ["Uttar Pradesh (Wheat)", "Madhya Pradesh (Rice & Wheat)"],
  "Horticulture": ["Himachal Pradesh (Apples)", "Maharashtra (Grapes)"],
  "Research": ["ICAR", "IARI", "NDRI", "IVRI", "CIFE", "CMFRI", "CIAE", "IIMR",
  "SBI", "CSSRI", "PAU", "ANGRAU", "UAS", "GBPUAT", "TNAU"] };
function buildPrompt(userQuestion) {
  let context = "You are an assistant. Only answer based on the following agriculture
categories and their places:\n";
  for (const [category, places] of Object.entries(availablePlaces)) {
    context += `- ${category}: ${places.join(", ")}\n`; }
  context += `\nQuestion: ${userQuestion}\nAnswer:`;
  return context; }
// Export for chatbot.js
window.buildPrompt = buildPrompt;
// ✔ Chatbot open/close
function toggleChatbot() {
  document.getElementById('chatbot-box').classList.toggle('active');
}
function closeChatbot() {
  document.getElementById('chatbot-box').classList.remove('active');
}
// ✔ Show first category by default
toggleContent('water');
</script>
```

EDUCATION SCRIPT:

```
<script>
// Toggle categories
```

```

function showContent(id) {
  document.querySelectorAll('.content-card').forEach(c => c.classList.remove('active'));
  document.getElementById(id).classList.add('active');
}
document.addEventListener("DOMContentLoaded", () => showContent('ancient'));
// Toggle chatbot
function toggleChatbot() {
  document.getElementById("chatbot-box").classList.toggle("active");
}
// Education context
const eduPlaces = {
  "Ancient Universities": ["Nalanda", "Takshashila", "Vikramashila"],
  "Modern Institutions": ["IIT Bombay", "IIT Delhi", "AIIMS", "ISI", "IIM Ahmedabad"]
};
function isEduQuery(q) {
  const keywords =
["education", "university", "college", "iit", "iim", "aiims", "school", "nalanda", "takshashila"];
  return keywords.some(k => q.toLowerCase().includes(k));
}
function buildPrompt(userMessage) {
  let context = "Answer only based on Indian Education system:\n";
  for (const [cat, items] of Object.entries(eduPlaces)) {
    context += `-${cat}: ${items.join(", ")}\n`;
  }
  context += `\nQuestion: ${userMessage}\nAnswer: `;
  return context;
}
// Chatbot logic
document.getElementById("chatbot-send").addEventListener("click", sendMessage);
document.getElementById("chatbot-input").addEventListener("keypress", e => {
  if (e.key === "Enter") sendMessage();
});
async function sendMessage() {
  const input = document.getElementById("chatbot-input");
  const msg = input.value.trim();
  if (!msg) return;
  const chat = document.getElementById("chatbot-messages");
  chat.innerHTML += `<div><b>You:</b> ${msg}</div>`;
  let payload = isEduQuery(msg) ? { message: buildPrompt(msg) } : { message: msg };
  try {
    const res = await fetch("http://localhost:5000/api/chat", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(payload)
    });
    const data = await res.json();
    chat.innerHTML += `<div><b>Bot:</b> ${data.reply}</div>`;
  } catch {
    chat.innerHTML += `<div><b>Bot:</b> ⚠ Could not connect to server.</div>`;
  }
}

```

```

    input.value = "";
    chat.scrollTop = chat.scrollHeight;
  }
</script>

```

HISTORY.SCRIPT:

```

<script>
// Show/hide content cards
function showContent(id) {
  document.querySelectorAll('.content-card').forEach(card => card.classList.add('hidden'));
  const el = document.getElementById(id);
  if (el) el.classList.remove('hidden');
}
// Chatbot toggle
function toggleChatbot() {
  document.getElementById("chatbot-box").classList.toggle("active");
}
// History chatbot context
const historyPlaces = {
  "Ancient": ["Indus Valley Civilization", "Vedic Period", "Mahajanapadas"],
  "Medieval": ["Cholas", "Rashtrakutas", "Rajputs", "Palas", "Kambaramayanam"],
  "Modern": ["Gandhi & Freedom Struggle", "Indian Railways", "Green Revolution",
"ISRO", "Digital India", "Metro Systems"]
};
function isHistoryQuery(userMessage) {
  const keywords = [
    "ancient", "vedic", "indus", "mauryan", "gupta",
    "mughal", "chola", "rashtrakuta", "rajput", "palas",
    "gandhi", "railway", "isro", "freedom", "modern india", "history"
  ];
  return keywords.some(k => userMessage.toLowerCase().includes(k));
}
function buildPrompt(userMessage) {
  let context = "Answer only based on Indian History categories:\n";
  for (const [cat, items] of Object.entries(historyPlaces)) {
    context += `- ${cat}: ${items.join(", ")}\n`;
  }
  context += `\nQuestion: ${userMessage}\nAnswer:`;
  return context;
}
document.addEventListener("DOMContentLoaded", () => {
  // Default open category
  showContent('ancient');
  const sendBtn = document.getElementById("chatbot-send");
  const inputEl = document.getElementById("chatbot-input");
  const chat = document.getElementById("chatbot-messages");
  async function sendMessage() {
    const msg = inputEl.value.trim();
    if (!msg) return;

```

```

chat.innerHTML += `<div><b>You:</b> ${msg}</div>`;
let payload;
if (isHistoryQuery(msg)) {
  payload = { message: buildPrompt(msg) };
} else {
  payload = { message: msg };
}
try {
  const response = await fetch("http://localhost:5000/api/chat", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(payload),
  });
  const data = await response.json();
  chat.innerHTML += `<div><b>Bot:</b> ${data.reply}</div>`;
} catch (err) {
  chat.innerHTML += `<div><b>Bot:</b> ⚠ Could not connect to server.</div>`;
}
inputEl.value = "";
chat.scrollTop = chat.scrollHeight;
}
sendBtn.addEventListener("click", sendMessage);
inputEl.addEventListener("keypress", (e) => {
  if (e.key === "Enter") sendMessage();
});
});
</script>

```

FEEDBACK.HTML:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Feedback | AI Guide</title>
  <link rel="stylesheet" href="css/styles.css" />
<style>
  body {
    margin: 0;
    padding: 0;
    font-family: 'Segoe UI', sans-serif;
    background: #f4f6f8;
    display: flex;
    flex-direction: column;
    align-items: center; }
  .header {
    background: linear-gradient(to right, #1565c0, #42a5f5);
    width: 100%;
    padding: 20px 0;
    text-align: center;

```



```

    color: white;
    font-size: 24px;
    font-weight: bold;
}
.feedback-card {
    background: #ffffff;
    margin-top: 40px;
    padding: 30px 40px;
    border-radius: 12px;
    box-shadow: 0 10px 25px rgba(0, 0, 0, 0.1);
    width: 400px;
}
.feedback-card h2 {
    text-align: center;
    color: #1565c0;
    margin-bottom: 20px; }
.form-group {
    margin-bottom: 20px; }
input, textarea {
    width: 100%;
    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 6px;
    font-size: 14px;
    background-color: #fafafa;
    transition: border-color 0.3s; }
input:focus, textarea:focus {
    border-color: #42a5f5;
    outline: none; }
textarea {
    resize: vertical;
    height: 120px; }
button {
    width: 100%;
    padding: 12px;
    background-color: #1976d2;
    border: none;
    color: white;
    font-size: 16px;
    border-radius: 6px;
    cursor: pointer;
    transition: background-color 0.3s ease;
}
button:hover {
    background-color: #0d47a1; }
.thank-you {
    display: none;
    text-align: center;
    color: green;
    margin-top: 15px;

```

```

    font-weight: bold; }
</style>
</head>
<body>
  <div class="header">
    AI Driven Deep Domain Navigator
  </div>
  <div class="feedback-card">
    <h2>Submit Your Feedback</h2>
    <form id="feedbackForm">
      <div class="form-group">
        <input type="email" id="email" placeholder="Your Email" required />
      </div>
      <div class="form-group">
        <textarea id="message" placeholder="Your feedback..." required></textarea>
      </div>
      <button type="submit">Send Feedback</button>
    </form>
    <div class="thank-you" id="thankYouMessage">✔ Thank you for your feedback!</div>
  </div>
  <script>
    const form = document.getElementById("feedbackForm");
    const thankYouMessage = document.getElementById("thankYouMessage");
    form.addEventListener("submit", async (e) => {
      e.preventDefault();
      const email = document.getElementById("email").value;
      const feedbackText = document.getElementById("message").value;
      try {
        const res = await fetch("http://localhost:5000/api/feedback", {
          method: "POST",
          headers: {
            "Content-Type": "application/json" },
          body: JSON.stringify({ email, feedbackText }) // ✔ Corrected key names });
        const data = await res.json();
        if (res.ok) {
          thankYouMessage.style.display = "block";
          form.reset();
        } else {
          alert("Error: " + data.message);
        }
      } catch (err) {
        alert("Server error");
        console.error(err);
      }
    });
  </script>
</body>
</html>

```